

Web Technologies

XML/HTML data processing

DOM
Document Object Model



Prof. Sabin Corneliu Buraga – profs.info.uaic.ro/sabin.buraga/
Assoc. Prof. Andrei Panu – profs.info.uaic.ro/andrei.panu/

“The golden rule is that there are no golden rules.”

George Bernard Shaw

-  how can we process the XML documents
-  details regarding the processing of XML/HTML documents via DOM
-  processing XML/HTML documents via DOM at Web client level
-  asynchronous data transfer between Web client(s) and server(s)



How can we process the XML documents?

preamble

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE html>
```

comment node

```
<!-- pentru redarea datelor, se recurge la o foaie de stiluri CSS -->
```

```
<?xml-stylesheet type="text/css" href="game.css" ?>
```

processing instruction

```
<!-- ... -->
```

```
<platform type="tablet">Android</platform>
```

```
<platform min-version="9" type="tablet">iOS</platform>
```

```
<platform min-version="10">
```

attribute node

```
<url>...</url>
```

```
<player>
```

start tag

end tag

```
<identity>
```

```
<first-name>Sabin</first-name>
```

```
<last-name>Buraga</last-name>
```

XML element marked
with a start tag and an
end tag

```
<!-- ... -->
```

```
</identity>
```

```
<points>30374</points>
```

```
</player>
```

text node
(characters)

```
</game>
```

Types of XML processing

manual processing

e.g., regular expressions 🤔

Types of XML processing

object-oriented processing

DOM (Document Object Model)

non-DOM

Types of XML processing

event-driven processing

SAX (Simple API for XML)

XPP (XML Pull Parsing)

details in S. Buraga, *Tehnologii XML*, Polirom, 2006

profs.info.uaic.ro/sabin.buraga/teach/courses/web/presentations/Sabin_Buraga-Tehnologii_XML.pdf

Types of XML processing

simplified processing

Simple XML

details in S. Buraga, *Tehnologii XML*, Polirom, 2006

profs.info.uaic.ro/sabin.buraga/teach/courses/web/presentations/Sabin_Buraga-Tehnologii_XML.pdf

Types of XML processing

specific processing

via specialized APIs in order to process
particular document types
e.g., KML, RSS, MathML, SVG,...

XML (parsers) processors **without validation**

verify only if the document is well formed

Expat, libxml, MSXML,...

XML (parsers) processors **with validation**

verify if the document is valid,
according to an XML validation method:
DTD – Document Type Definition, XML Schema

Apache Xerces, JAXP, libxml, MSXML,...



Details regarding the processing of XML/HTML documents via DOM

DOM model

introduction
specification levels
DOM interfaces
DOM Core
implementations
examples

DOM in the Web browser

dom: intro

Goal:

object-oriented – standardized – processing
of XML and/or HTML documents

dom: characterization

An abstract API for XML/HTML

platform- and language-independent

standardized by the Web Consortium

dom: characterization

Defines a logical tree-like structure
of XML documents

document = hierarchy of a set of objects

based on which we can access/modify XML nodes and data

dom: specification levels

DOM 1 (1998), DOM 2 (2001),
DOM 3 (2004), DOM 4 (2015)

DOM Core for XML

DOM HTML for standardized processing of
Web pages – usually, on client-side (browser)

dom: specification levels

DOM Living Standard

dom.spec.whatwg.org

HTML specific – current version

in continuous development
(latest update: 15 March 2026)

Markup to test ([permalink](#), [save](#), [upload](#), [download](#), [hide](#)):

```
<!DOCTYPE html>
<html><head><title>DOM</title></head><body><article><h1 class="h">Web</h1>
Document <em>Object</em> Model</article></body>
```

DOM view ([hide](#), [refresh](#)):

```
└─ DOCTYPE: html
  └─ HTML
    └─ HEAD
      └─ TITLE
        └─ #text: DOM
    └─ BODY
      └─ ARTICLE
        └─ H1 class="h"
          └─ #text: Web
        └─ #text:
          └─ IMG src="image" alt="Web"
        └─ #text: Document
        └─ EM
          └─ #text: Object
        └─ #text: Model
```

in the HTML case, the element names
are available in capitals

an HTML document and its DOM tree
represented via **Live DOM Viewer**
software.hixie.ch/utilities/js/live-dom-viewer/

Markup to test ([permalink](#), [save](#), [upload](#), [download](#), [hide](#)):

```
<!DOCTYPE html>
<html><head><title>DOM</title></head><body><article><h1 class="h">Web</h1>
Document <em>Object</em> Model</article></body>
```

[DOM view](#) ([hide](#), [refresh](#)):

```
DOCTYPE: html
HTML
├── HEAD
│   └── TITLE
│       └── #text: DOM
├── BODY
│   └── ARTICLE
│       ├── H1 class="h"
│       │   └── #text: Web
│       ├── #text:
│       ├── IMG src="image" alt="Web"
│       ├── #text: Document
│       ├── EM
│       │   └── #text: Object
│       └── #text: Model
```

Why does this
node appear?

an HTML document and its DOM tree
represented via **Live DOM Viewer**
software.hixie.ch/utilities/js/live-dom-viewer/

dom: interfaces

An abstract manner to access and modify
an XML document's internal representation

data is encapsulated by objects,
hidden and/or protected from direct external processing

dom: interfaces

Does not imply a concrete, specific implementation

DOM provides implementation-independent
interfaces for data access/processing

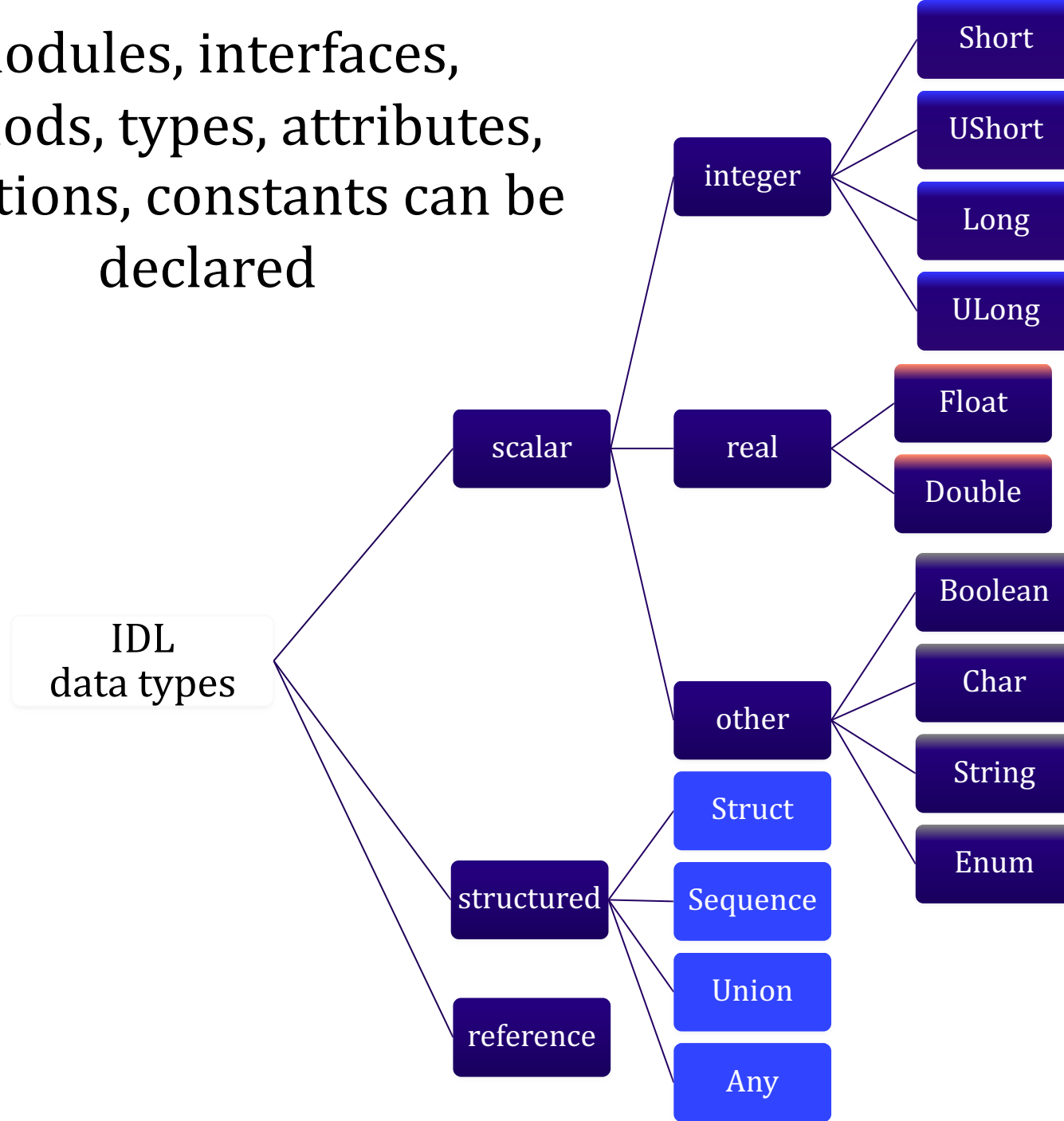
dom: interfaces – IDL

These interfaces are defined via
IDL (Interface Description Language)

defines object types by specifying their interfaces
(properties + public methods)

current specification (2018): www.omg.org/spec/IDL/

modules, interfaces,
methods, types, attributes,
exceptions, constants can be
declared



dom: interfaces – WebIDL

WebIDL – official specification for HTML
part of HTML Living Standard (27 March 2026)

webidl.spec.whatwg.org

dom: interfaces – example

Declaring the interface regarding the list of nodes
(elements, attributes, characters, comments,...)
of the XML tree: **NodeList**

```
interface NodeList {  
    getter Node? item (unsigned long index);  
    readonly attribute unsigned long length;  
};
```

dom: interfaces – example

```
interface NodeList {  
  getter Node? item (unsigned long index);  
  readonly attribute unsigned long length;  
};
```

method for retrieving info
regarding a node in a list of
nodes

input: an unsigned long int
parameter

result: a value of **Node** type

a read-only property
of type unsigned long integer

dom: interfaces – example

Specifying the interface regarding an XML
attribute: **Attr**

```
interface Attr : Node {  
  readonly attribute DOMString name;  
  readonly attribute boolean specified;  
  attribute DOMString value;  
};
```

dom: interfaces – example

```
interface Attr : Node {  
  readonly attribute DOMString name;  
  readonly attribute boolean specified;  
  attribute DOMString value;  
};
```

Attr extends Node



3 properties



name designates the name of the XML attribute

specified indicates whether the attribute was actually specified

value represents the current value of the attribute

dom: core

A document $\equiv$ hierarchy of **node-objects** that can implement (specialized) interfaces

nodes have descendants or are leaf nodes

tree traversal is performed in preorder,
depth-first – **tree order**

dom: core

The access to data
– node lists, attributes, values,... –
is performed by using specific methods
for each node type of the tree

dom: core – node types

Document	Element, ProcessingInstruction, Comment, DocumentType
Document Fragment	Element, ProcessingInstruction, Comment, Text, CDATASection,...
Element	Element, Text, Comment, CDATASection, EntityReference,...
Attr	Text, EntityReference
Text	– (leaf node of the DOM tree)

dom: core

Fundamental interfaces:

DOMException

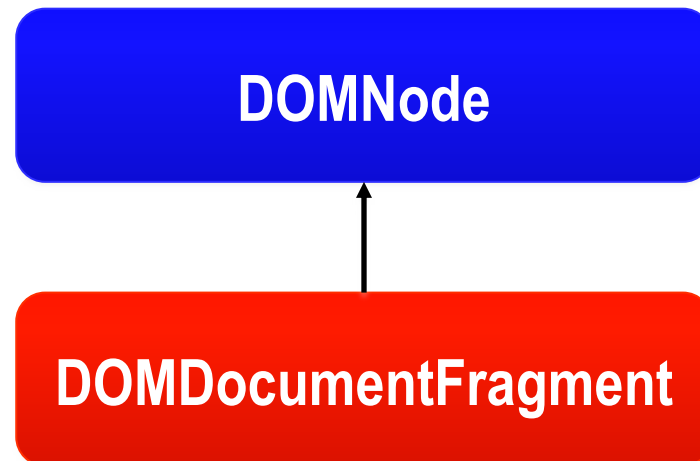
manages the processing exceptions

dom: core

Fundamental interfaces:

DocumentFragment : **Node**

access to tree fragments
(minimal object without a root node)



dom: core

Fundamental interfaces:

Document

provides document access
for reading and/or changing

dom: core

Fundamental interfaces:

Document

useful properties:

`doctype` – indicates the document type (DTD)

`documentElement` – specifies the root element

Essential methods of the **Document** interface

createElement() – creates a node of type element

createTextNode() – creates a node of type text (actual content)

createAttribute() – creates a node of type attribute (**Attr**)

getElementsByTagName() – provides the list of elements
having a specific name (tag)

getElementById() – provides the element
identified by the value of the **id** attribute

importNode() – imports a node from another document
by cloning the data of this node

etc.

dom: core

Fundamental interfaces:

Node

allows access to tree nodes

node types

– e.g., **Document**, **Element**,

CharacterData (**Text**, **Comment**, **CDATASection**),

DocumentFragment, **DocumentType**, **EntityReference**, ... –
processed in a unitary manner

Important properties of the **Node** interface

nodeName – provides the name of the current node

nodeValue – offers the current value of the node

nodeType – provides the type of the node: element,
attribute,...

parentNode – designates the parent node of the current node

parentElement – specifies the parent element of the node

Node types (nodeType property)	Value
ELEMENT_NODE	1
ATTRIBUTE_NODE	2
TEXT_NODE	3
CDATA_SECTION_NODE	4
ENTITY_REFERENCE_NODE	5
ENTITY_NODE	6
PROCESSING_INSTRUCTION_NODE	7
COMMENT_NODE	8
DOCUMENT_NODE	9
DOCUMENT_TYPE_NODE	10
DOCUMENT_FRAGMENT_NODE	11
NOTATION_NODE	12

Important properties of the **Node** interface

childNodes – provides the list of child nodes of the current node

firstChild – designates the first child of the current node

lastChild – specifies the last child of the current node

nextSibling – indicates the next node (sibling)
on the same level as the current node

previousSibling – represents the previous node (sibling)
on the same level as the current node

etc.

Useful methods provided by the **Node** interface

insertBefore() – inserts a node before the current one

cloneNode() – clone the data of the current node

compareDocumentPosition() – compare the position of the current node with another node in the document

isEqualNode() – test whether the current node is equivalent to another node (same node type, belongs to the same namespace, has the same value, both nodes have the same number of children, each child node has the same index for both nodes, etc.) – also see **isSameNode()**

contains() – checks if the node includes a specific node

Useful methods provided by the **Node** interface

hasChildNodes() – checks if the current node has child nodes

appendChild() – add a child node

replaceChild() – replace a child node

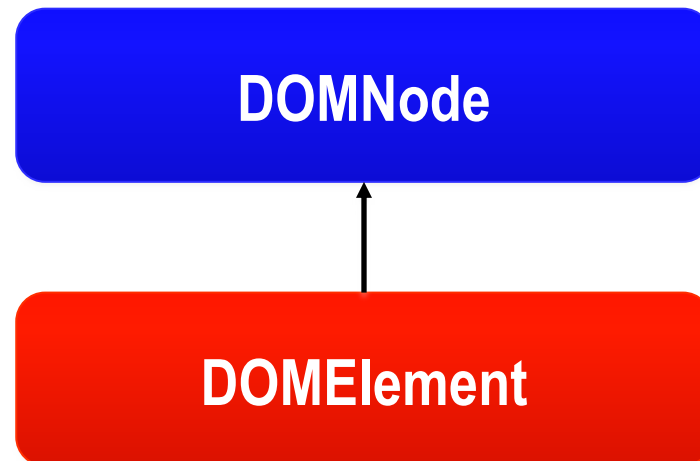
removeChild() – removes a child node

dom: core

Fundamental interfaces:

Element

facilitates the access to XML elements



Useful properties of the **Element** interface

tagName – provides the name of the element (start-tag)

id – specifies the value of the identifier
(the value of the **id** attribute)

childNodes – provides the list of child nodes of the element

firstChild **lastChild** – indicates the first/last child node

attributes – provides the list of attributes associated to the
element

nextElementSibling – indicates the next element (sibling) on
the same level in the tree as the current element

previousElementSibling – idem, but for the preceding one

Useful methods specified by the **Element** interface

getAttribute() – provides the value of an attribute associated to the current element

getAttributeNode() – provides the node of type attribute

setAttribute() – sets the value of an attribute

removeAttribute() – removes an attribute associated with the element

hasAttribute() – checks the existence of a specific attribute

Useful methods specified by the **Element** interface

getAttribute() – provides the value of an attribute associated to the current element

getAttributeNode() – provides the node of type attribute

setAttribute() – sets the value of an attribute

removeAttribute() – removes an attribute associated with the element

hasAttribute() – checks the existence of a specific attribute

to process attributes belonging to a given XML namespace,
NS-suffixed methods will be used

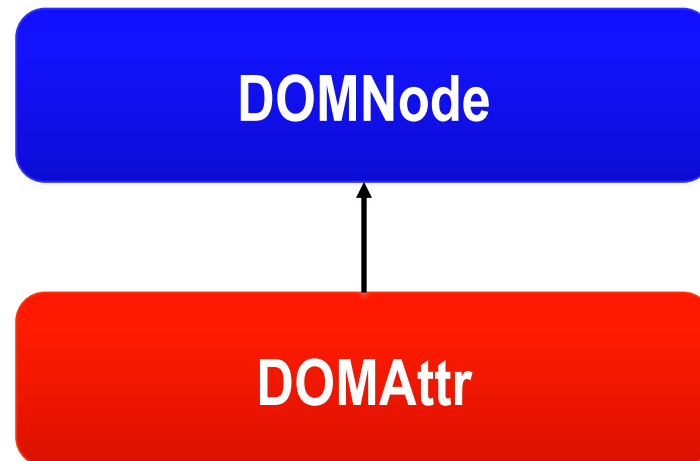
e.g., **getAttributeNS()** **setAttributeNS()** etc.

dom: core

Fundamental interfaces:

Attr : Node

access to the attributes of an element



dom: core

Fundamental interfaces:

NodeList – access to collections of nodes through indexes
(vector)

NamedNodeMap – access based on keys (associative array)
in case of HTML, for attribute lists

dom: core

Fundamental interfaces:

NodeList + NamedNodeMap

property: **length** – the length of the list of nodes

methods: **item()** **getNamedItem()** **getNamedItemNS()**
setNamedItem() **removeNamedItem()** and others

dom: core

Other interfaces of interest:

- ParentNode** – the parent node specific to a node
- ChildNode** – the child node belonging to a node
- Elements** – collection of nodes of type **Element**

dom: access to nodes – xpath

 Accessing data from a document

XPath 1.0 – W3C Recommendation (1999)

later versions – less often used:

XPath 2 (2010), XPath 3 (2014), XPath 3.1 (2017)

www.w3.org/TR/xpath/all/

dom: access to nodes – xpath

XPath expression – the basic construct

evaluation depends on the context:

a node of the XML document

position

a function (predefined or programmer-defined)

a namespace declaration

dom: access to nodes – xpath

XPath expression – the basic construct

after expression evaluation, an object is returned:

a node-set

a Boolean value – true/false

a number (a floating-point number)

a string

dom: xpath – operators

Descendent /

/html/body/article

/game/platform

dom: xpath – operators

Recursive traversal //

//platform

//div/a

child nodes are traversed in the order given by the DFS
(depth-first search) algorithm

dom: xpath – operators

Wildcard *

selects all child elements of the node in the current context
(context node)

/game/*/identity

player/*

Săptămâna 1 (16 - 20 februarie)

#web ★ film proiecte eval () biblio contact

★ săptămâna 1

- Chestiuni organizatorice / Organizational Matters [EN] Lecture 600 x 80.1
- Curs: World Wide Web – aspecte arhitecturale / World Wide Web – Architectural Aspects [EN] concepts, client/server model, hypertext, Web resource, URI, Web site, Web application
- Demo: The Mother of All Demos
- Resurse: The History of the Web, The birth of the Web (CERN)
- Laborator: Limbajul de marcare HTML

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Ap

Search HTML +

```
<!DOCTYPE html>
<html lang="ro">
  <head>
  <body id="top" itemscope itemtype="http://schema.org/WebPage">
    <div class="nav">
    <article>
      <header>
      <div class="info">
        <p>
        </div>
        <p>
        <p>
        <p>
        <nav>
```

```
<section id="week1">
  <h2 class="film">
    <ul>
      <li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture0">
      <li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture1">
      <li class="demo">
```

html > body#top > article > section#week1

Filter Output Errors W

```
>> $x("/html/body/*/section")
← Array(9) [ section#week1, section#week2, section#week3, section#week4, section#week5, section#week6, section#week7, section#week8, section ]
```

/html/body/*/section
the evaluation of XPath
expressions in Web browser's
console via the **\$x** object

dom: xpath – operators

Current node .

./player

dom: xpath – operators

Attribute @

platform/@min-version

table/@*

dom: xpath – operators

Filter/index []

/game/platform[2]
//li[@class = "lab"]

general form: [*expression*]

```
▼ <section id="week1">
  ▶ <h2>...</h2>
  ▼ <ul>
    ▶ <li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture0">...</li>
    ▶ <li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture1">...</li>
    ▶ <li class="bib">...</li>
    ... ▼ <li class="lab"> == $0
      ▼ <p>
        "Laborator: "
        <a href="https://www.w3.org/html/" title="Detalii despre HTML (HyperText Markup Language)">Limbajul de
        marcare HTML</a>
        " - de studiat "
        ▶ <span itemscope itemtype="http://schema.org/CreativeWork">...</span>
        " și "
        <a href="https://www.html-5-tutorial.com/" title="Un tutorial privitor la HTML">HTML5 - A Tutorial for
        Beginners</a>
        </p>
        ▶ <ul>...</ul>
      
```

html body article div#past section#week1 ul li.lab

Console

Filter Default levels ☒ Group similar

```
> $x ('//li[@class="lab"]')
< ▼ (13) [li.Lab, li.Lab, li.Lab, li.Lab, li.Lab, li.Lab, li.Lab, li.Lab, li.Lab, li.Lab, li.Lab, li.Lab, li.Lab] ⓘ
```

//li[@class = "lab"]

13 nodes(Element items) having the string "lab" as the value of the **class** attribute

dom: xpath – operators

For logical and numerical values,
the common operators could be used:

or and = != <= < >= >
+ - * div mod

string comparison is *case sensitive*

dom: xpath – axes

Current node (<i>context node</i>)	self
Parent node	parent
Children nodes	child
Descendant nodes	descendant
Attribute nodes	attribute
Namespace nodes	namespace

dom: xpath – axes

Ascendant nodes

ancestor

Ascendant nodes
or current node

ancestor-or-self

Descendant nodes
or current node

descendant-or-self

```
<game>
  <title>Angry Profs</title>
  <platform type="tablet">Android</platform>
  <platform min-version="9" type="tablet">iOS</platform>
  <platform min-version="10">Windows</platform>
  <url>...</url>
  <player>
    <identity>
      <first-name>Sabin</first-name>
      <last-name>Buraga</last-name>
      <!-- ... -->
    </identity>
    <points>30374</points>
  </player>
</game>
```

using the **preceding** axis to access all the above nodes
a Web testing tool: www.freeformatter.com/xpath-tester.html

XPath expression

```
/game/player/identity/preceding::*
```

xsl: xpath – axes

The result of the XPath expression
`/game/player/identity/preceding::*`

```
Element='<title>Angry Profs</title>'  
Element='<platform type="tablet">Android</platform>'  
Element='<platform min-version="9" type="tablet">iOS</platform>'  
Element='<platform min-version="10">Windows</platform>'  
Element='<url>...</url>'
```


dom: xpath – short syntax

self	≡	.
parent	≡	..
attribute	≡	@
namespace	≡	:

../first-name

//child::*[@min-version > 9]

dom: xpath – functions

Nodes: `id()`, `position()`, `count()`, `name()`,
`namespace-uri()`, `last()`, ...

Node types: `node()`, `text()`, `comment()`,
`processing-instruction()`

Strings: `concat()`, `starts-with()`, `contains()`, `substring()`,
`string-length()`, `translate()`, ...

Booleans: `not()`, `true()`, `false()`

Numbers: `sum()`, `round()`, `floor()`, `number()`, ...

counting the nodes satisfying a given condition

count(/game/platform[@min-version >= 9])

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <game>
3   <title>Angry Profs</title>
4   <platform type="tablet">Android</platform>
5   <platform min-version="9" type="tablet">iOS</platform>
6   <platform min-version="10">Windows</platform>
7   <url>...</url>
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL XPath Results ▼

XPath Query: `count(/game/platform[@min-version >= 9])`

[Result]: 2

XML Tools extension for Visual Studio Code

dom: xpath – functions

`count(//a[@href]) = count(//a)` ▶ true

all **<a>** elements include the **href** attribute

`count(//li[@class]) = count(//li)` ▶ false

there are **** elements without **class** attribute

```
> $x ('count(//a[@href]) = count(//a)')
```

```
< true
```

```
> $x ('count(//li[@class]) = count(//li)')
```

```
< false
```

```

▶ <section id="week4">...</section>
▼ <section id="week5">
  ▶ <h2>...</h2>
  ▼ <ul>
    ▶ <li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture5">...</li>
    ▼ <li class="demo">
      ▼ <p> == $0
        "Demo: "
        <a href="demos/php/php.zip" title="Arhivă .zip">Exemple
        programme PHP</a>
        " (arhivă "
        <code>zip</code>
        ", 41.5K)"
        </p>
      </li>
    ▶ <li class="bib">...</li>
    ▶ <li class="tools">...</li>
  
```

`//*[contains(text(), "Demo")]`
getting all nodes containing
the **"Demo"** string

html body article div#past section#week5 ul li.demo **p**

Styles Event Listeners DOM Breakpoints Properties Accessibility

⋮ Console

▶ 🔍 top ▼ Filter Default levels ▼ ☒ Group similar

> \$x ('//*[contains(text(), "Demo")]')

◀ ▼ (14) [p, p, p, **p**, p, p, p, p, p, p, p, p, p, p] ⓘ

```
<projects>
  <project>
    <title>Super</title>
    <desc>A super project for super students!</desc>
    <stud>3</stud>
    <url>http://super.info/</url>
  </project>

  <project>
    <title>Super--</title>
    <desc>A not so super project...</desc>
    <stud>2</stud>
  </project>
</projects>
```

the average number of students assigned per project:

`sum(/projects/project/stud/text()) div count(/projects/project/stud)`

XPath Query: `sum(/projects/project/stud/text()) div count(/projects/project/stud)`

[Result]: 2.5

evaluating XPath expressions
via command line interface

```
<projects>
  <project>
    <title>Super</title>
    <desc>A super project for super students!</desc>
    <stud>3</stud>
    <url>http://super.info/</url>
  </project>

  <project>
    <title>Super--</title>
    <desc>A not so super project...</desc>
    <stud>2</stud>
  </project>
</projects>
```

xmllint tool (uses **LibXML**)

provided by GNU/Linux distributions + macOS

```
xmllint projects-dtd.xml --xpath "substring(/projects/project[1]/desc/text(), 4, 7)"
```

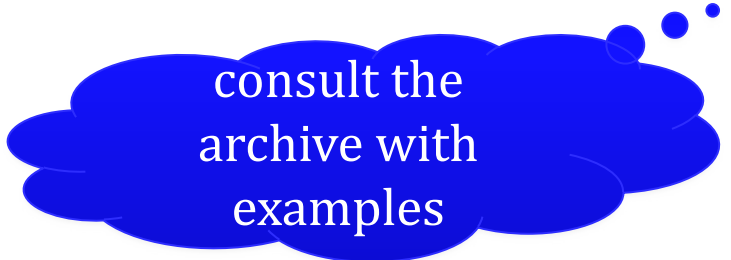

(instead of) break



dom: xpath – programming

Example:

using XPath for processing an RSS news feed in a PHP program which uses the **DOMDocument**, **DOMXPath**, **DOMNode** classes



consult the
archive with
examples

```

<rss xmlns:dc="http://purl.org/dc/elements/1.1/" version="2.0">
  <channel>
    <title>InfoQ</title>
    <link>https://www.infoq.com</link>
    <description>InfoQ feed</description>
    <item>
      <title>TigerFS Mounts PostgreSQL Databases as a Filesystem for Developers...</title>
      <link>https://www.infoq.com/news/2026/04/tigerfs-postgresql-filesystem/</link>
      <description>&lt;p&gt; TigerFS is a new experimental filesystem that mounts a
database as a directory and stores files directly in PostgreSQL....&lt;/p&gt;</description>
      <category>Database</category>
      <category>Linux</category>
      <category>Development</category>
      <category>AI, ML &amp; Data Engineering</category>
      <category>Open Source</category>
      <pubDate>Sat, 04 Apr 2026 08:18:00 GMT</pubDate>
      <guid>https://www.infoq.com/news/2026/04/tigerfs-postgresql-filesystem/</guid>
      <dc:creator>Renato Losio</dc:creator>
      <dc:date>2026-04-04T08:18:00Z</dc:date>
      <dc:identifier>/news/2026/04/tigerfs-postgresql-filesystem/en</dc:identifier>
    </item>
    <!-- alte noutăți... (i.e., elemente <item>) -->
  </channel>
</rss>

```

the general structure of a
RSS feed can be studied at

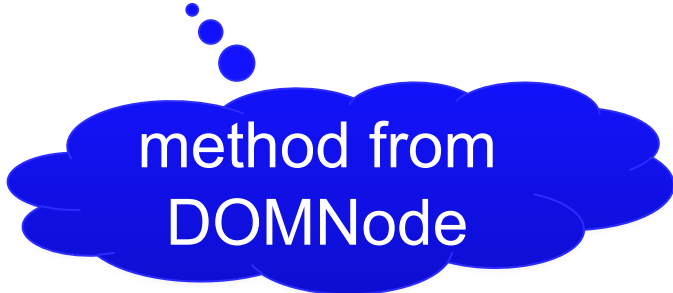
www.rssboard.org

```

define ('FEED', 'https://feed.infoq.com/'); // RSS feed address
define ('XPATH', '/rss/channel/item');      // used XPath expression

try {
    $dom = new DomDocument ();
    $dom->load (FEED); // loading the XML document
    $xpath = new DOMXPath ($dom);
    // go through each result (here, the <item> elements)
    foreach ($xpath->query(XPATH) as $news) {
        // get the title of each news: the value of the <title> element
        echo '<p>Noutate: <em>'.
            $news->getElementsByTagName('title')->item(0)->nodeValue . '</p>';
        echo '<p>Calea nodului: <code>'.
            $news->getElementsByTagName('title')->item(0)->getNodePath() .
            '</code></p>';
    }
}
catch (RuntimeException $e) {
    echo $e->getMessage ();
}

```



method from
DOMNode

Date RSS de la InfoQ

Noutate: *Pinterest Reduces Spark OOM Failures by 96% Through Auto Memory Retries*

Calea nodului: [/rss/channel/item\[1\]/title](#)

Noutate: *Presentation: Duolingo's Kubernetes Leap*

Calea nodului: [/rss/channel/item\[2\]/title](#)

Noutate: *Article: A Better Alternative to Reducing CI Regression Test Suite Sizes*

Calea nodului: [/rss/channel/item\[3\]/title](#)

Noutate: *Podcast: Context Engineering with Adi Polak*

Calea nodului: [/rss/channel/item\[4\]/title](#)

Noutate: *Dynamic Languages Faster and Cheaper in 13-Language Claude Code Benchmark*

Calea nodului: [/rss/channel/item\[5\]/title](#)

Noutate: *Java News Roundup: TornadoVM 4.0, Google ADK for Java 1.0, Grails, Tomcat, Log4j, Gradle*

Calea nodului: [/rss/channel/item\[6\]/title](#)

Noutate: *Anthropic Designs Three-Agent Harness Supports Long-Running Full-Stack AI Development*

Calea nodului: [/rss/channel/item\[7\]/title](#)

Noutate: *TigerFS Mounts PostgreSQL Databases as a Filesystem for Developers and AI Agents*

Calea nodului: [/rss/channel/item\[8\]/title](#)

The **query()** method of the class **DOMXPath** provides the list of nodes determined by evaluating the XPath expression (an instance of **NodeList** type is returned)

To get results of a different type

- for example, the evaluation of an XPath expression can be of type integer –, the method **evaluate()** is used

```
$items = $xpath->query (XPATH);  
echo '<p>Total apariții de elemente <code>&lt;item&gt;</code>:</p>';  
echo '<p>Folosind proprietatea length (DOMNodeList): ' .  
  $items->length . '</p>';  
echo '<p>Folosind funcția XPath count(): ' .  
  $xpath->evaluate("count(" . XPATH . ")") . '</p>';
```

count(/rss/channel/item)

Total apariții de elemente <item>:

Folosind proprietatea length (DOMNodeList): 15

Folosind funcția XPath count(): 15

dom: implementations

DOMDocument – PHP class

php.net/manual/en/book.dom.php



Dom\Document – new DOM implementation (PHP 8.4+)

php.net/manual/en/class.dom-document.php

details: wiki.php.net/rfc/dom_additions_84

evaluating Xpath expressions: **Dom\XPath** class

php.net/manual/en/class.dom-xpath.php

for HTML: **Dom\HTMLDocument**

php.net/manual/en/class.dom-htmldocument.php

dom: implementations

HXT (Haskell XML Toolbox) – DOM processing in Haskell
wiki.haskell.org/HXT

JAXP (Java Architecture for XML Processing)
docs.oracle.com/javase/tutorial/jaxp/

QDOM – from Qt (C++): doc.qt.io/qt-6/qdomdocument.html

libxml – XML processor written in C: xmlsoft.org
foundation for C++, Perl, PHP, Python, Ruby,... libraries

org.w3c.dom.Document –DOM interface (Android)
developer.android.com/reference/org/w3c/dom/Document.html

dom: implementations

Xerces DOM API – project focused on XML processing
(C++ and Java): xerces.apache.org

XmlDocument – .NET Framework class (C# et al.)
learn.microsoft.com/en-us/dotnet/api/system.xml.xmldocument

XMLDocument – Objective-C / Swift class
developer.apple.com/documentation/foundation/xmldocument

xml.dom – Python module
docs.python.org/3/library/xml.dom.html

dom: specific apis (examples)

node-xmpp – Node.js (JavaScript) library for XMPP (*Extensible Messaging and Presence Protocol*)
github.com/xmppjs/xmpp.js

OpenDocument Format (ODF) SDKs – ODF document processing in C#, Java, PHP, Python,...
www.oasis-open.org/tc-opendocument/

Open Street Map API – access to XML data provided by the OpenStreetMap cartographic free service
wiki.openstreetmap.org/wiki/OSM_XML

dom: specific apis (examples)

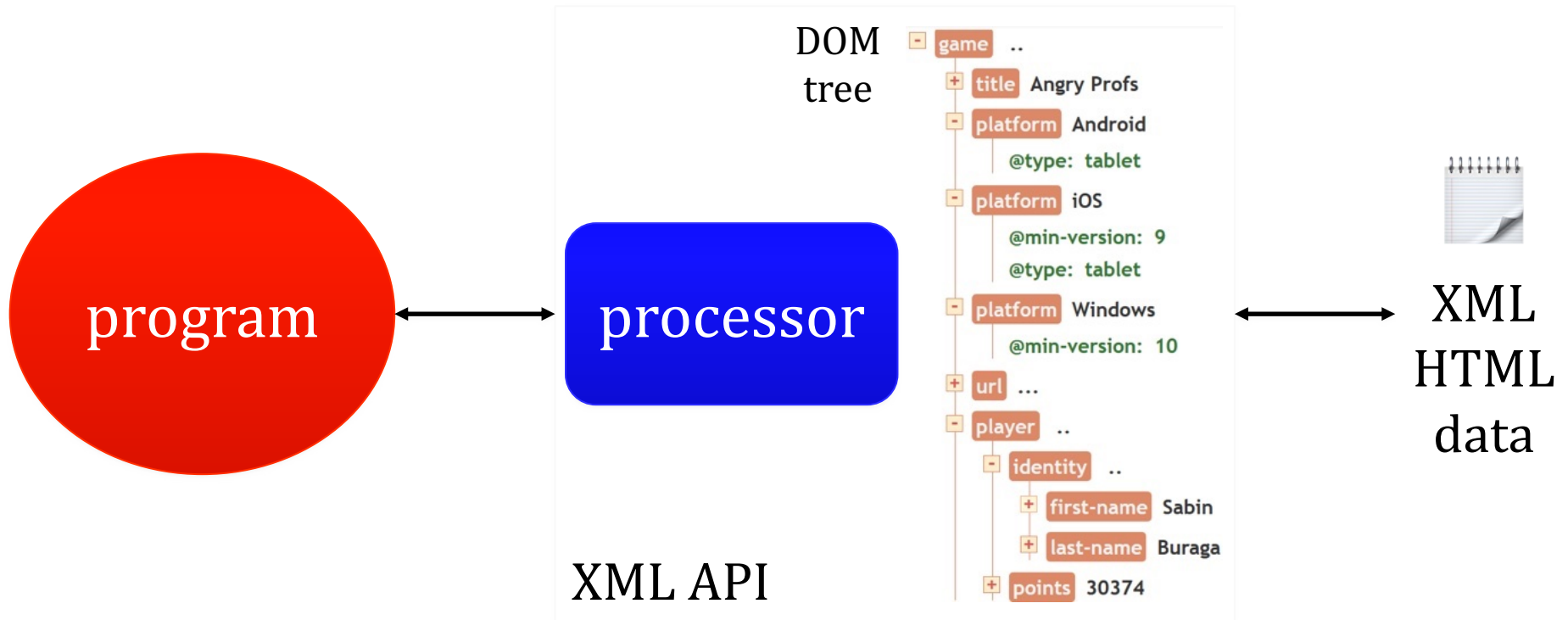
SVG DOM API – SVG document processing in Java (Apache Batik): xmlgraphics.apache.org/batik/
developer.mozilla.org/en-US/docs/Web/API/SVG_API

Twiml (*Twilio Markup Language*) API – accessing Twilio telephony services via C#, Java, Node.js, PHP, Python, Ruby libraries: www.twilio.com/docs/voice/twiml

.NET MAUI (*Multi-platform App UI*) – specifying user-interfaces for cross-platform applications (iOS, Android, macOS, Windows) via C# and XAML:
dotnet.microsoft.com/apps/maui

dom: implementations

XML/HTML document processing





Case study:
processing - in PHP -
the information about
the list of project proposals

study the code
examples associated
to this lecture

the processed XML document:

```
<?xml version="1.0" ?>
<!-- XML document about project proposals -->
<projects>
  <project class="M">
    <title>MegaProj</title>
    <desc>...</desc>
    <stud>2</stud>
    <url>http://web.info/</url>
  </project>
  <project>
    <title>Web4all</title>
    <desc />
    <stud>3</stud>
  </project>
</projects>
```

```
$modified = 0; // indicates whether the DOM tree has been modified
```

```
$dom = new DomDocument; // instantiate a DOM object
```

```
$dom->load("projects.xml"); // load the XML document
```

```
$projs = $dom->getElementsByTagName("project");
```

```
foreach ($projs as $proj) { // process every <project>
```

```
// get all <title> elements and select the first one
```

```
$titles = $proj->getElementsByTagName("title");
```

```
echo "<p>Project: <em>" . $titles[0]->nodeValue . "</em>";
```

```
if ($proj->hasAttribute("class")) { // is the project's class specified?
```

```
echo " of class " . $proj->getAttribute("class") . "</p>";
```

```
} else {
```

```
echo " of unknown class</p>";
```

```
// the attribute "class" does not exist, we are creating it – the project
```

```
// will be of class D
```

```
$attr = $proj->setAttribute("class", "D");
```

```
$modified = 1; // mark the tree as modified
```

```
}
```

```
}
```

```
if ($modified) { // if the DOM has been modified, display its textual representation
```

```
$xmldoc = $dom->saveXML(); // serialization
```

```
echo "<pre>" . htmlentities($xmldoc) . "</pre>";
```

```
}
```

DOM processing
in PHP language

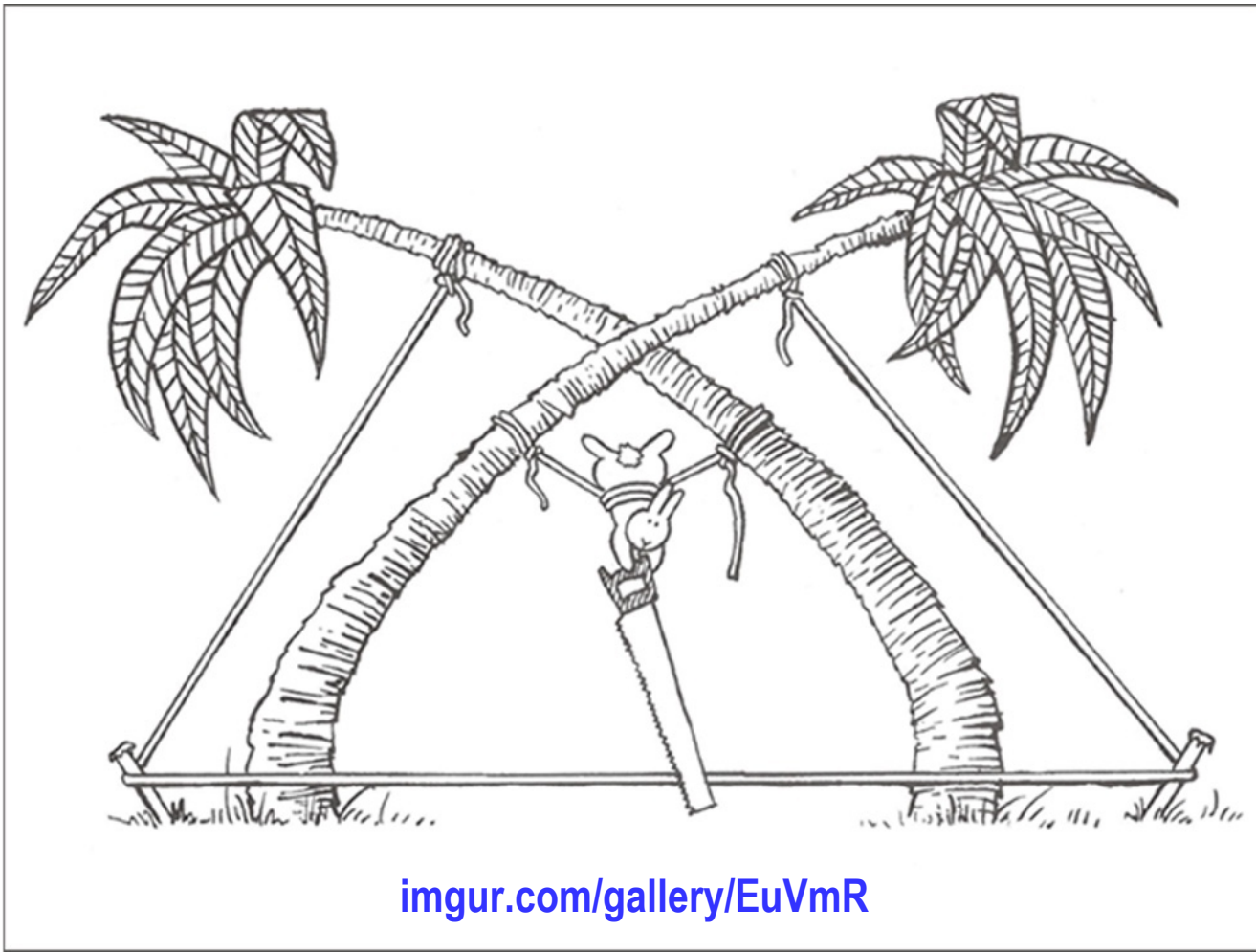
the result obtained after DOM processing
done by the PHP program:

Project: *MegaProj* of class M

Project: *Web4all* of unknown class

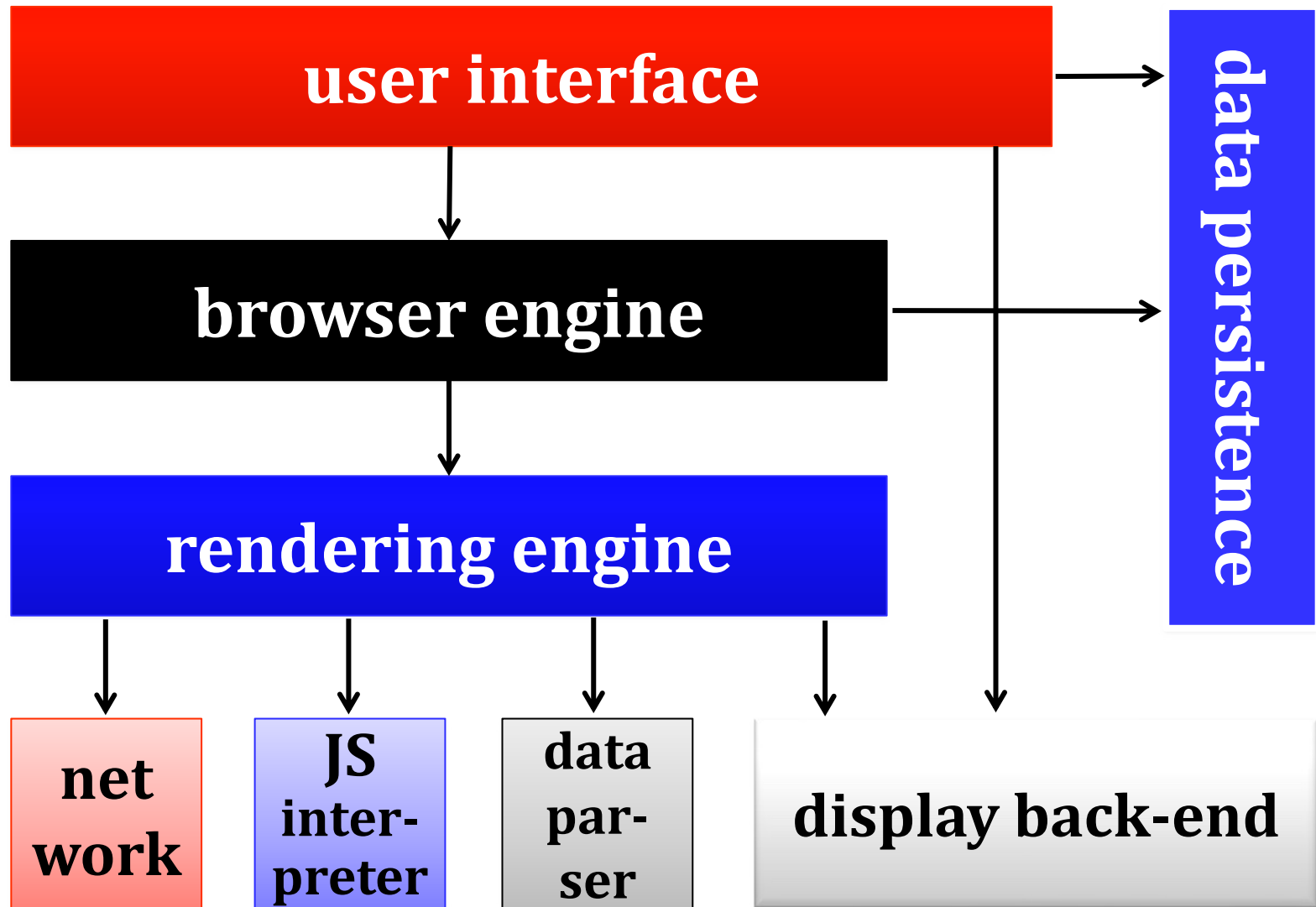
```
<?xml version="1.0"?>
<!-- XML document about project proposals -->
<projects>
  <project class="M">
    <title>MegaProj</title>
    <desc>...</desc>
    <stud>2</stud>
    <url>http://web.info/</url>
  </project>
  <project class="D">
    <title>Web4all</title>
    <desc/>
    <stud>3</stud>
  </project>
</projects>
```


🌵 XML documents processing through DOM in a Web client?



imgur.com/gallery/EuVmR

The architecture of a Web browser



generic + particular XML
processing: SVG, RSS,...

dom: browser

The accessing/processing of HTML and XML documents
– without validation – is performed
via DOM by using JavaScript (ECMAScript) programs
interpreted by the Web browser

A JavaScript program has access to the **DOM** (**Document Object Model**) tree corresponding to the HTML document

mainly **DOM Living Standard** is used

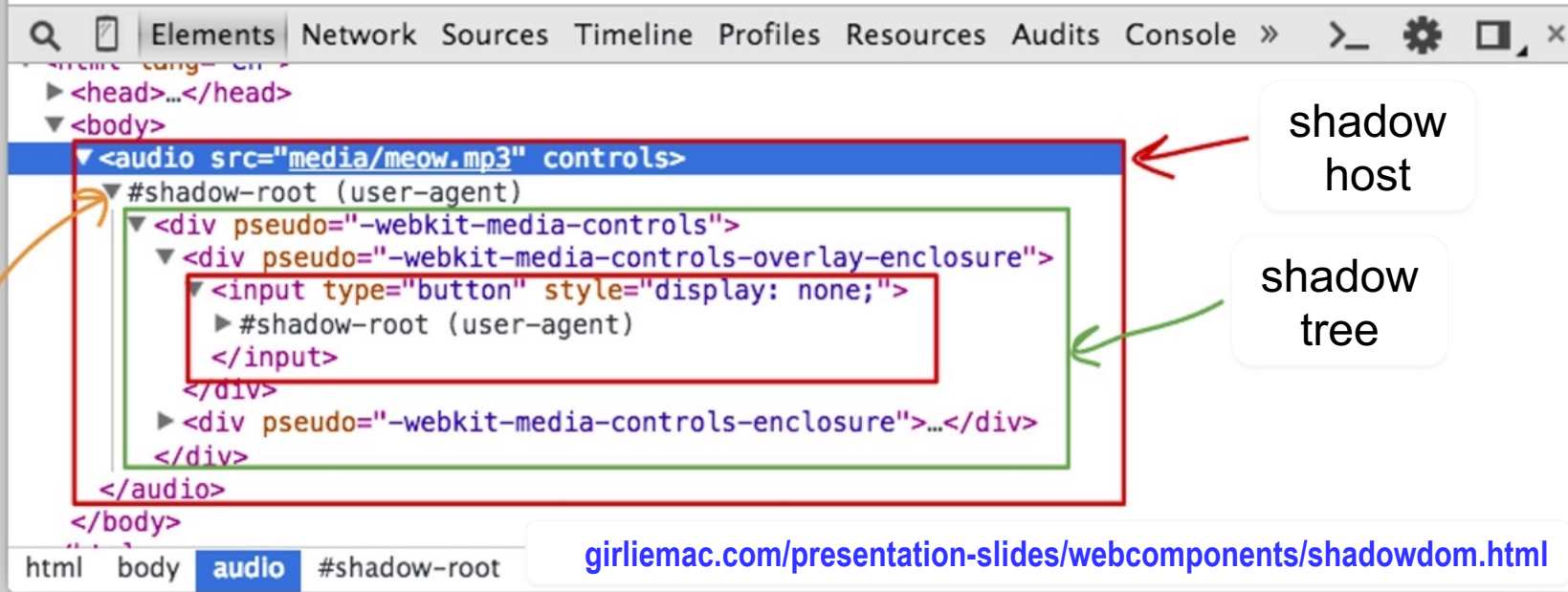
HTML5 specific, in continuous development

The JS program uses objects which implement DOM interfaces to access and change the internal representation of an HTML document (i.e. the DOM tree)

Shadow DOM (shadow tree)

The DOM tree whose root element is a node (shadow root) of another tree corresponding to a document

always attached to a tree of nodes
(cannot exist on its own,
but only encapsulated by a document)



shadow DOM

described in DOM Living Standard (15 March 2026)

dom.spec.whatwg.org/#shadow-trees

developer.mozilla.org/Web/Web_Components/Using_shadow_DOM

JavaScript programs also have access to various objects offered by the execution environment provided by the browser

e.g., information regarding the execution context (browser's characteristics, network latency), browsing history, content display window, (a)synchronous data transfer,...

	CSSOM	 30	 69			✓	advanced		CSS
	CSSOM View	 61	 88			✓	  		CSS
	data URLs	 0	 1			✓	  	HTTP and addressing	
	DOM	 9	 307			✓	  		Core
	DOM Parsing and Serialization	 54	 10			✓	  		Core
	ECMAScript	 0	 641			✓	  		JavaScript
	ECMAScript Internationalization API	 15	 80			✓	  		JavaScript
	Encoding	 0	 18			✓	  		Core
	Encrypted Media Extensions	 19	 37			✓	  		Media
	Event Timing API	 7	 9			✓	  	Performance/profiling	
	Fetch	 27	 56			✓	  		Core
	Fetch Metadata Request Headers	 22	 4			✓	  		Security
	File and Directory Entries API	 78	 22			✓	  	Storage and files	
	File API	 140	 44			✓	  	Storage and files	

technologies provided by browser – html-now.github.io
also explore caniuse.com

The DOM tree associated with the HTML document can be accessed/altered via the **document** object

instance of the class that implements the **HTMLDocument** interface

```

interface HTMLDocument : Document {
    attribute DOMString title;           // document title
    readonly attribute DOMString referrer; // URL of the referring resource
    readonly attribute DOMString domain; // document domain
    readonly attribute DOMString URL;    // absolute URL of the document
    attribute HTMLElement body;       // access to <body> element

    attribute DOMString cookie;          // access to cookies
    // emits an exception if a value is assigned

    void open (); // opens a writing stream (alters the current DOM)
    void close (); // closes writing flow & forces content rendering
    void write (in DOMString text); // writes a string (e.g., HTML markups)
    void writeln (in DOMString text); // idem, but insert also a new line
    // provides a list of elements according to the name attribute
    NodeList getElementsByName (in DOMString numElement);
};

```

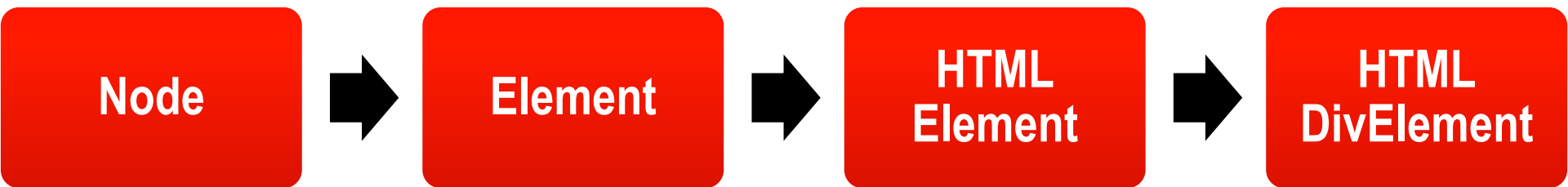
interface specified in the **WebIDL** declarative language

Because the **HTMLDocument** interface
extends **Document**,
we can use the functionalities stipulated by the
general DOM specification

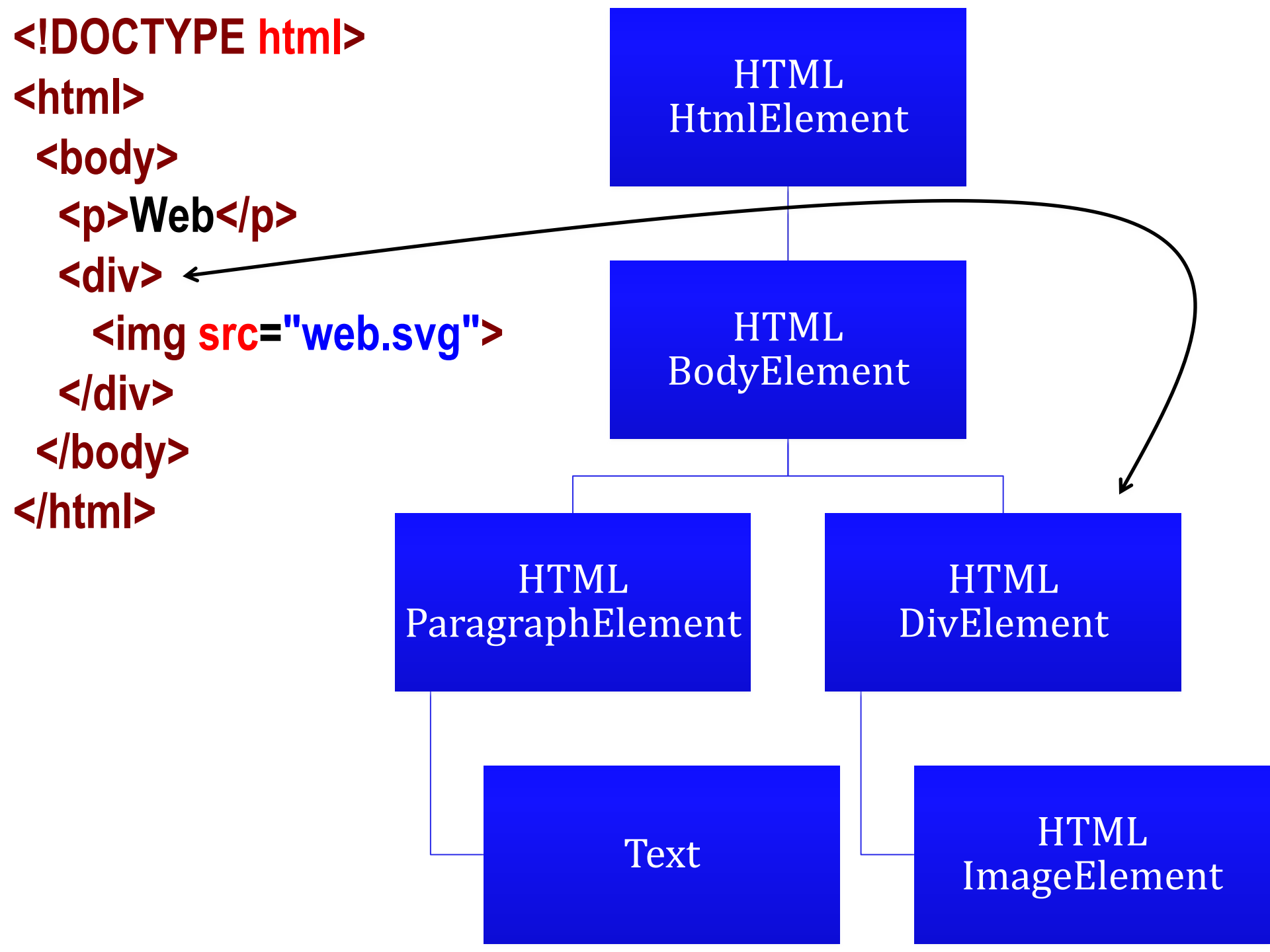
for example, the **documentElement** property
designates the root node of an HTML document

HTMLElement interface extends the general interface provided by DOM

each specific HTML element derives from it



an interface specific to each HTML element



```

// a generic HTML element
interface HTMLElement : Element {
    attribute DOMString    id;                // identifier associated to this element
    attribute DOMString    title;             // explanatory title
    attribute DOMString    lang;              // content language
    attribute DOMString    className;         // CSS class name used for rendering
};

// specifying a Web form
interface HTMLFormElement : HTMLElement {
    readonly attribute HTMLCollection elements; // HTML elements included by this form
    readonly attribute long                length; // number of form fields
    attribute DOMString                    action; // URI of the resource that process the data
    attribute DOMString                    enctype; // MIME type used for data encoding
                                                // (e.g., application/x-www-form-urlencoded)
    attribute DOMString                    method; // used HTTP method (GET or POST)
    void                                  submit(); // sends data to URI specified by 'action'
};

// an image (raster graphical content)
interface HTMLImageElement : HTMLElement {
    attribute DOMString    alt;                // alternative text describing the content
    attribute DOMString    src;                // URL of the image resource
};

```

The **HTMLCollection** interface models a list of nodes

A node can be accessed using a numeric index or based on an identifier (e.g., set via the **id** attribute)

```
interface HTMLCollection {  
  // represents the length of the list  
  readonly attribute unsigned long length;  
  // offers a node via a numeric index  
  Node    item (in unsigned long index);  
  // provides a node based on the name  
  Node    namedItem (in DOMString name);  
};
```


DOM HTML specific properties:

innerHTML

(mutable) property providing the HTML code
of an **Element** node

DOM HTML specific properties:

textContent

property that gets/sets the textual content of a node and its possible descendants

HTML ▼

```
1 <form id="unFormular">
2   <!-- corpul formularului va fi generat via JavaScript -->
3 </form>
```

JavaScript + No-Library (pure JS) ▼

```
1 function adaugaControl (element, tip, nume, explicatie) {
2   // vom crea un element <input> atasandu-i atributule esentiale
3   let control = document.createElement ('input');
4   control.type = tip;
5   control.id = nume;
6   control.name = nume;
7
8   // vom adauga si o eticheta explicativa
9   let eticheta = document.createElement ('label');
10  eticheta.htmlFor = nume;
11  eticheta.innerHTML = explicatie;
12  // vom altera diverse stiluri CSS
13  eticheta.style.background = 'lightgray';
14  eticheta.style.color = '#333';
15  eticheta.style.padding = '0.2em';
16
17  // vom plasa elementele <label> si <input> intr-un container
18  let container = document.createElement ('div');
19  container.className = "interactiv";
20  container.appendChild (eticheta);
21  container.appendChild (control);
22
23  // vom adauga nodul container la elementul dat ca parametru
24  element.appendChild (container);
25 }
26
27 // preluam elementul de tip formular
28 let formular = document.getElementById ('unFormular');
29 // adaugam controalele de interactiune dorite
30 adaugaControl (formular, 'text', 'nume', 'Numele meu:
```

CSS ▼

```
1 .interactiv {
2   border-style: dotted;
3   border-width: 1px;
4   border-color: gray;
5   width: 20em;
6   padding: 1em;
7   margin: 0.2em;
8   font-family: sans-serif;
9 }
10 label {
11   width: 50%;
12   display: block;
13   float: left;
14   margin-right: 0.3em;
15 }
```

JS Fiddle

Numele meu: Codul secret: Particip? ☒Web Hackathon 2021 ☐Web Hackathon 2022 ☐

dynamic JavaScript generation of
a web form

jsfiddle.net/busaco/0wvn3fha/

```
function addControl (element, type, name, explanation) {  
    // create an <input> element attaching its essential attributes  
    let control = document.createElement ('input');  
    control.type = type;  
    control.id = name;  
    control.name = name;  
  
    let label = document.createElement ('label');    // an explanatory label...  
    label.htmlFor = name;  
    label.innerHTML = explanation;  
    label.style.background = 'lightgray';           // ...plus various CSS styles  
    label.style.color = '#333';  
    label.style.padding = '0.2em';  
  
    // place the <label> and <input> elements in a container  
    let container = document.createElement ('div');  
    container.className = "interactiv";  
    container.appendChild (label);  
    container.appendChild (control);  
  
    // add the container node to the element specified as parameter  
    element.appendChild (container);  
}
```

```
// get the element of type form
let form = document.getElementById ('unFormular');

// add the desired interaction controls
addControl (form, 'text', 'nume', 'Numele meu: ');
addControl (form, 'password', 'parola', '<em>Codul secret</em>: ');
addControl (form, 'checkbox', 'particip', 'Particip? ');
for (let year = 2021; year <= 2025; year++) {
    addControl (form, 'radio', 'year', 'Web Hackathon ' + year);
}
```

```
<form id="unFormular">
  <!--corpul formularului va fi generat via JavaScript-->
  <div class="interactiv">
    <label for="nume" style="background: lightgray none repeat scroll 0% 0%; color: rgb(51, 51, 51);
padding: 0.2em;">Numele meu:</label>
    <input id="nume" type="text" name="nume">
  </div>
  <div class="interactiv"> ... </div>
  <div class="interactiv"> ... </div>
  <div class="interactiv"> ... </div>
  <div class="interactiv"> ... </div>
  <div class="interactiv"> ... </div>
</form>
```

the DOM tree corresponding to
the HTML code generated by
the program

Support for processing CSS properties

based on a specific object model

CSSOM (CSS Object Model)

draft specification – 23 February 2026

drafts.csswg.org/cssom/

Support for processing CSS properties

currently, changing the style properties
can be done via the **HTMLElement.style** property

```
// associating several CSS styles  
elem.style.cssText = "color: blue; border: 1px solid #000";  
// similar to the line – has the effect of altering the DOM tree:  
elem.setAttribute ("style", "color: blue; border: 1px solid #000;");
```

developer.mozilla.org/Web/API/CSSStyleDeclaration

Finding nodes via CSS selectors

Selectors API

initially, W3C recommendation (2013)

currently, part of the DOM Living Standard (2026)

Finding nodes via CSS selectors

querySelector (*selectors*)

provides the first element – using pre-order depth traversal – which matches the group of selectors (comma delimited)

querySelectorAll (*selectors*)

provides the list of type NodeList of all elements found

Finding nodes via CSS selectors

```
// all <li> elements selected via CSS (data of type NodeList)
let elements = document.querySelectorAll ("ul.menu > li");
for (var i = 0; i < elements.length; i++) {
  process (elements.item (i));    // process each node
}
```

★ săptămâna 6

- Curs: **Programare Web – Dezvoltarea aplicațiilor Web în PHP** (continuare)

Web, programming, development, application server, PHP, tools, examples

- Demo: **Exemple de programe PHP** (arhivă zip, 45K)
- Resurse: **PDO Tutorial, Modern PHP without a Framework, PHP Framework Interop Group** , **PHP—The Wrong Way**
- Curs: **Modelarea datelor. Familia XML. SVG**

Inspector Console Debugger Network Style Editor >>

Run { } ^ v 🔍 ✕ 🗑️ Filter Output

```
1 document.querySelectorAll
  ("section[id^=\"week\"] :nth-
2 | child(odd) > h2");
```

selectorii CSS3

Errors Warnings Logs Info Debug CSS XHR Requests

```
>> document.querySelectorAll ("section[id^=\"week\"] :nth-
child(odd) > h2");
```

← ▼ NodeList(3) [h2.film , h2.film , h2.film]

- ▶ 0: <h2 class="film">
- ▶ 1: <h2 class="film">
- ▶ 2: <h2 class="film">

length: 3

example – use the Web browsers's console:

```
document.querySelector ("section[id^=\"week\"] :nth-child(odd) > h2");
```

Event handling

defining callbacks to be executed when an event occurs

event = action triggered within the runtime environment
that can conduct to a reaction from a program

Event handling

the JavaScript code invoked when an event occurs can be encapsulated in a function that handles the event (event handler)

Event handling

traditionally, JavaScript code to be executed is attached
when a basic event occurs

(e.g., **onclick**, **onmouseover**, **onchange**, **onload**, **onkeypress**,...)
on an element

Event handling

```
<button onclick="alert ('Auch!');">Press me!</button>
```

Event handling

```
<button onclick="alert ('Auch!');">Press me!</button>
```

to inhibit the execution of the default action,
the JavaScript code must return *false*

Event handling

more „evolved” via DOM 1:

```
document.getElementById (“identifier”).onclick  
= treatClick;
```

Event handling

standardized event handling:
minimum, according to the **DOM Level 2 Events** spec

www.w3.org/TR/DOM-Level-2-Events/

Event handling

specifying activities performed when an event occurs

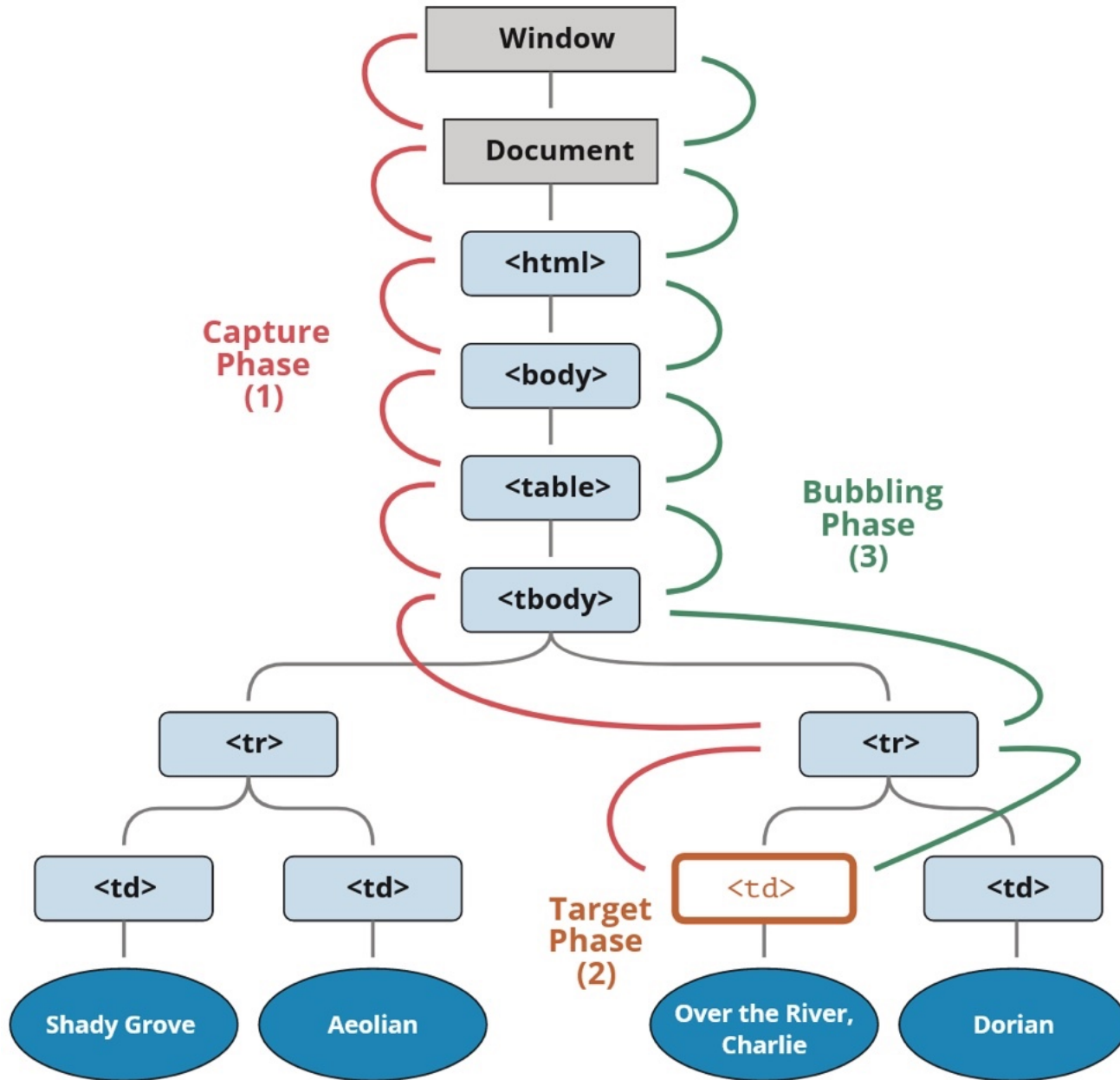
object.addEventListener (“event”, function, mode);

Event handling

hierarchical description of the event flow

capture versus bubble

see javascript.info/bubbling-and-capturing



study the tutorial *Introduction to events*

developer.mozilla.org/docs/Learn/JavaScript/Building_blocks/Events

Event handling

hierarchical description of the event flow

mode = true

try to handle the event starting from the root to the target
object – **capture phase**

Event handling

hierarchical description of the event flow

mode = false

try to handle the event when it is propagated from the
object where it occurred to its superior entities –
bubbling phase

Event handling

a standard set of events will be used

Event types – user interaction

mouse: **click dblclick mouseenter mousedown mouseup mouseover
mousemove contextmenu select wheel**

touch: **touchstart touchend touchcancel touchmove**

keyboard: **keypress keydown keyup**

clipboard: **copy cut paste**

view: **resize scroll fullscreenchange fullscreenerror**

drag & drop: **dragstart drag dragenter dragover
dragleave dragend drop**

form: **focus blur select submit reset**

document (page lifecycle): **load DOMContentLoaded
beforeunload unload abort cancel**

Event types – regarding resources

network: **online offline**

session history: **pagehide pageshow popstate**

printing: **beforeprint afterprint**

media: **canplay play playing pause suspend waiting seeking
seeked stalled complete ended emptied durationchange
ratechange volumechange timeupdate ...**

progress: **loadstart progress error timeout abort load loaded**

storage: **change storage**

Web socket: **open message error close**

Event types – others

CSS transitions: **transitionstart transitionrun
transitionend transitioncancel**

CSS animations: **animationstart animationend
animationiteration**

value change: **broadcast hashchange input readystatechange...**

for details, also explore
developer.mozilla.org/Web/Events

Event types – specific to modern HTML

associated to the available APIs:

Barcode Detection, Bluetooth, Canvas, Clipboard, Device Memory, Fetch, File System Access, Fullscreen, Geolocation, Image Capture, IndexedDB, Media Capture & Streams, Notifications, Page Visibility, Payment Request, Push, Sensor, Streams, Touch Events, URL, Vibration, Web Animations, Web Audio, WebGL, WebRTC, Web Workers, WebSocket, Web Storage, WebXR Device etc.

developer.mozilla.org/docs/Web/API

Event handling

useful properties of the **Event** object

type

specifies the event's type as string
e.g., **"click"**, **"load"**, **"scroll"**, **"submit"**

Event handling

useful properties of the **Event** object

target

designates the node
on which the event was initially triggered

Event handling

useful properties of the **Event** object

currentTarget

indicates the node that handles the event

Event handling

useful properties of the **Event** object

bubbles

indicates whether the event is propagating
towards ascending elements (*true* value)
or towards descendants (*false* value)

Event handling

useful properties of the **Event** object

bubbles

e.g., abort, error, select, submit, resize, scroll,
click, mousedown, mouseover, mousemove, mouseout,
touchstart, touchend events can have **bubbles** = *true*

Event handling

useful properties of the **Event** object

bubbles

in case of the **focus**, **blur**, **load**, **unload** events,
the **bubbles** property has the value *false*

Event handling

useful properties of the **Event** object

cancelable

specifies whether the event can be interrupted

Event handling

useful properties of the **Event** object

cancelable

for example, for the **load**, **unload**, **abort**, **error**, **select**, **focus**, **blur**,
resize, **scroll**, **touchcancel** events,
the **cancelable** property is set as *false*

Event handling

useful properties of the **Event** object

cancelable

for events like **click**, **mousedown**, **mouseup**,
mouseover, **mousemove**, **mouseout**, **touchstart**, **touchend**,
touchmove the **cancelable** property can be *true*

Event handling

eliminating the handling of an event

removeEventListener ()

Event handling

ignoring the default behavior

preventDefault ()

Event handling

sending the event to be processed
according to the model offered by the implementation

dispatchEvent ()

Event handling

stopping the propagation of an event
within the DOM tree

stopPropagation ()

```
<html>
<head><title>Evenimente JS</title></head>
<body>
  <form>
    <textarea id="editor"></textarea>
    <input type="text" id="text" />
  </form>
</body>
<script type="application/javascript">
const infoEv = e => { console.log (
  `Eveniment: ${e.type}\n
  Character: ${String.fromCharCode (e.charCode)}\n
  Element țintă al evenim.: ${e.target}\n
  Ținta curentă: ${e.currentTarget}`);
};
document.addEventListener("keypress", infoEv);
</script>
</html>
```

example: displaying data regarding an event

advanced

b

Inspector Console Debugger Network DOM

Filter Output Errors Warnings Logs Info Debug CSS XHR Requests

Eveniment: keypress

keypress.html:17:31

Character: W

Element țintă al evenim.: [object HTMLTextAreaElement]

Ținta curentă: [object HTMLDocument]

key press in the
<textarea> field

Eveniment: keypress

keypress.html:17:31

Character: e

Element țintă al evenim.: [object HTMLInputElement]

Ținta curentă: [object HTMLDocument]

key press in the <input>
field

Eveniment: keypress

keypress.html:17:31

Character: b

Element țintă al evenim.: [object HTMLBodyElement]

Ținta curentă: [object HTMLDocument]

key press in the body
of the Web page

// adaptation after https://eloquentjavascript.net/14_event.html

```
const trateazaEveniment = ev => {  
  // place a 'dot' at the coordinates of the mouse cursor  
  let pct = document.createElement('div');  
  pct.className = (ev.type === 'dblclick') ? 'punct roz' : 'punct';  
  pct.style.left = (ev.pageX - 5) + 'px';  
  pct.style.top = (ev.pageY - 5) + 'px';  
  document.body.appendChild(pct);  
  console.log(` ${ev.type}: Am plasat un punct la coord.  
              (${ev.pageX}, ${ev.pageY}).`);  
};
```

see the full example
in the archive

// "ascultăm" evenimentele click și dblclick

```
document.addEventListener('click', trateazaEveniment);  
document.addEventListener('dblclick', trateazaEveniment);
```

example: handling the **click** and **dblclick** events

Un click (dublu) de mouse, te rog...

div.punct | 10x10

Elements

Console

Sources

Network

Performance

Memory

Application

»

⋮

✕

<!DOCTYPE html>

<html>

▶<head>...</head>

...▼<body> == \$0

<p>Un click (dublu) de mouse, te rog...</p>

▶<script type="application/javascript">...

</script>

<div class="punct" style="left: 311px; top: 107px;"></div>

<div class="punct" style="left: 143px; top: 181px;"></div>

<div class="punct" style="left: 143px; top: 181px;"></div>

<div class="punct roz" style="left: 143px; top: 181px;"></div>

<div class="punct" style="left: 95px; top: 71px;"></div>

<div class="punct" style="left: 694px; top: 22px;"></div>

</body>

html

body

Styles

Computed

Event Listeners

»

🔄

☒ Ancestors

All

▼

☒ Framework listeners

▼ click

▼ document [clicks.html:25](#)

useCapture: false

passive: false

once: false

▼ handler: ev => {...}

▶ [[Scopes]]: Scopes[2]

▶ [[FunctionLocation]]: <unknown>

▶ __proto__: f ()

name: "trateazaEveniment"

length: 1

caller: (...)

arguments: (...)

▶ dblclick

Remark:

modern Web browsers support the handling of non-standard + programmer-defined events

CustomEvent

dom.spec.whatwg.org/#interface-customevent
developer.mozilla.org/Web/API/CustomEvent

The screenshot displays the browser's developer tools with three panels:

- HTML Panel (Left):** Shows the document structure. The `<body>` element is selected, showing its attributes like `logged-out`, `env-production`, and `page`. It also shows nested `<div>` elements.
- Event Listeners Panel (Middle):** A list of events attached to the selected element. The events are:
 - `DOMContentLoaded` (from `.../github/sessions/two-factor.ts:111`)
 - `dragend` (from `...ages/has-interactions/has-interactions.ts:97`)
 - `dragenter` (from `...de_modules/delegated-events/dist/index.js:78`)
 - `dragover` (from `...de_modules/delegated-events/dist/index.js:78`)
 - `error` (from `...s.com/assets/packages/failbot/failbot-error.ts:4`)
 - `file-attachment-accept` (from `...ents/dist/index.js:78`)
 - `file-attachment-accepted` (from `...ts/dist/index.js:78`)
 - `filter-input-updated` (from `...-events/dist/index.js:78`)
 - `filterable:change` (from `...ted-events/dist/index.js:78`)
 - `focus` (from `...ssets.com/assets/packages/onfocus/onfocus.ts:22`)
 Each entry shows the event name, source file, and propagation mode (Bubbling or Capturing).
- Event Listener Code Panel (Bottom):** Shows the code for the selected `focus` event:


```

f "focus" event listener code
let e = t.target;
if (e instanceof HTMLElement && e.nodeType !== Node.DOCUMENT_NODE)
  for (let t of r.matches(e)) t.data.call(null, e)
      
```

inspecting – via the tools included in the browser – the standard + programmer-defined events (only those written in capital letters), plus the propagation mode: *bubbling versus capturing*

practical example: GitHub

dom: additional resources

Marijn Haverbeke, *Eloquent JavaScript (4th Edition)*, 2024
– the “*The Document Object Model*” chapter
eloquentjavascript.net/14_dom.html

Ilya Kantor, *The Modern JavaScript Tutorial*, 2026
– the “*DOM Tree*” section
javascript.info/dom-nodes

EventTarget
developer.mozilla.org/Web/API/EventTarget

dom: custom attributes

Attributes defined by the programmer
(custom HTML5 attributes)

in an HTML document custom attributes can be included,
which are prefixed with **data-** and manipulated via

HTMLElement.dataset

can store private values
available at Web page/application level

developer.mozilla.org/Learn/HTML/Howto/Use_data_attributes

```
<div id="person" data-id="69" data-user="tux" data-date-of-birth>  
Tuxy Pinguinnesscool</div>
```

```
<script>  
let elem = document.querySelector('#person'); // get data  
// elem.id == 'person'  
// elem.dataset.id === '69'  
// elem.dataset.dateOfBirth === '' (date of birth was not specified)  
elem.dataset.dateOfBirth = '2001-04-01'; // set the date of birth  
// include another custom attribute set in the program  
elem.dataset.size = 'big';  
</script>
```

```
<div id="person" data-id="69" data-user="tux" data-date-of-birth>  
Tuxy Pinguinnesscool</div>
```

```
<script>  
let elem = document.querySelector('#person'); // get data  
// elem.id == 'person'  
// elem.dataset.id === '69'  
// elem.dataset.dateOfBirth === '' (date of birth was not specified)  
elem.dataset.dateOfBirth = '2001-04-01'; // set the date of birth  
// include another custom attribute set in the program  
elem.dataset.size = 'big';  
</script>
```

```
>> elem  
← ▶ <div id="person" data-id="69" data-user="tux" data-date-of-birth="2001-04-01" data-size="big">  
>> elem.dataset.dateOfBirth  
← "2001-04-01"
```

use of the Web browser's console to access and modify
data- attributes using **dataset**

(instead of) break





What is the way to transfer data
asynchronously
between Web client(s) and server(s)?

web interaction: ajax

Enabling asynchronous data transfer between an HTML document displayed in the client (browser) and an application running on a Web server

AJAX – Asynchronous JavaScript and XML

(Jeese James Garrett)

via the XMLHttpRequest object and/or Fetch API

web interaction: ajax

A suite of open technologies:

standardized languages for data structuring – usually,
HTML – and presentation: CSS

web interaction: ajax

A suite of open technologies:

view + interaction on the Web client (browser)
via the DOM standard

web interaction: ajax

A suite of open technologies:

exchange and manipulation of data represented through:

JSON

various XML dialects (e.g., RSS, KML,...)

HTML

other formats – e.g., CSV

web interaction: ajax

A suite of open technologies:

processing using the ECMAScript (JavaScript) language

web interaction: ajax

A suite of open technologies:

(a)synchronous data transfer via HTTP
facilitated by the XMLHttpRequest object
available via JavaScript in the Web browser

HTML Living Standard, 15 March 2026

xhr.spec.whatwg.org

web interaction: ajax

Basic component: **XMLHttpRequest** object

can send HTTP requests – e.g., GET, POST,... –
from a program running on client (browser)
to a Web application/service existing on a server,
in an **asynchronous** or **synchronous** way

web interaction: ajax

Basic component: XMLHttpRequest object

it is not required anymore for the Web pages to be completely reloaded, their content – structured via HTML – being manipulated with DOM in the browser, in accordance with the data received from server

web interaction: ajax

Important methods provided by XMLHttpRequest

open ()

initiates – opens – an HTTP connection to the server,
issuing a request: GET, POST,...

web interaction: ajax

Important methods provided by XMLHttpRequest

send ()

(asynchronously) transmits data – e.g., JSON, XML, etc. –,
to the application/service running on server

web interaction: ajax

Important methods provided by XMLHttpRequest

send ()

(asynchronously) transmits data – e.g., JSON, XML, etc. –,
to the application/service running on server

any listener (associated to the **loadstart**, **progress**, **load**,
loadend, **timeout**, **abort**, **error** events)
should be set before sending data

web interaction: ajax

Important methods provided by XMLHttpRequest

abort ()

cancels the current data transfer

web interaction: ajax

Important methods provided by XMLHttpRequest

setRequestHeader ()

specifies various HTTP header fields

examples: Cookie, Keep-Alive, User-Agent,...

web interaction: ajax

Important methods provided by XMLHttpRequest

getResponseHeader ()

provides the value of a certain field from the header of the HTTP response message sent by the server

web interaction: ajax

Important methods provided by **XMLHttpRequest**

getAllResponseHeaders ()

provides all the HTTP fields sent by the server,
except **Set-Cookie**

web interaction: **ajax**

Basic properties of **XMLHttpRequest**

readyState

provides the code of transfer state:

- 0 – **UNSENT**
- 1 – **OPENED**
- 2 – **HEADERS_RECEIVED**
- 3 – **LOADING**
- 4 – **DONE**

web interaction: ajax

Basic properties of XMLHttpRequest

status

provides the HTTP code returned by the Web server:

200 (Ok)

404 (Not Found)

500 (Internal Server Error)

...

web interaction: ajax

Basic properties of XMLHttpRequest

statusText

contains the message corresponding to the HTTP status code provided by the Web server

web interaction: ajax

Basic properties of XMLHttpRequest

responseText
responseXML

stores the response (data) obtained from server

web interaction: ajax

Basic properties of XMLHttpRequest

onreadystatechange

specifies the function to be called at each change of state regarding data transfer between server and client



a handler for data transfer
events

web interaction: ajax

Exceptions that could be emitted

AbortError
InvalidAccessError
InvalidStateError
NetworkError
SecurityError
TimeoutError

...

Other features:

the data transmitted can be of several types
(**ArrayBuffer**, **Blob**, **Document**, **DOMString**, **FormData**)

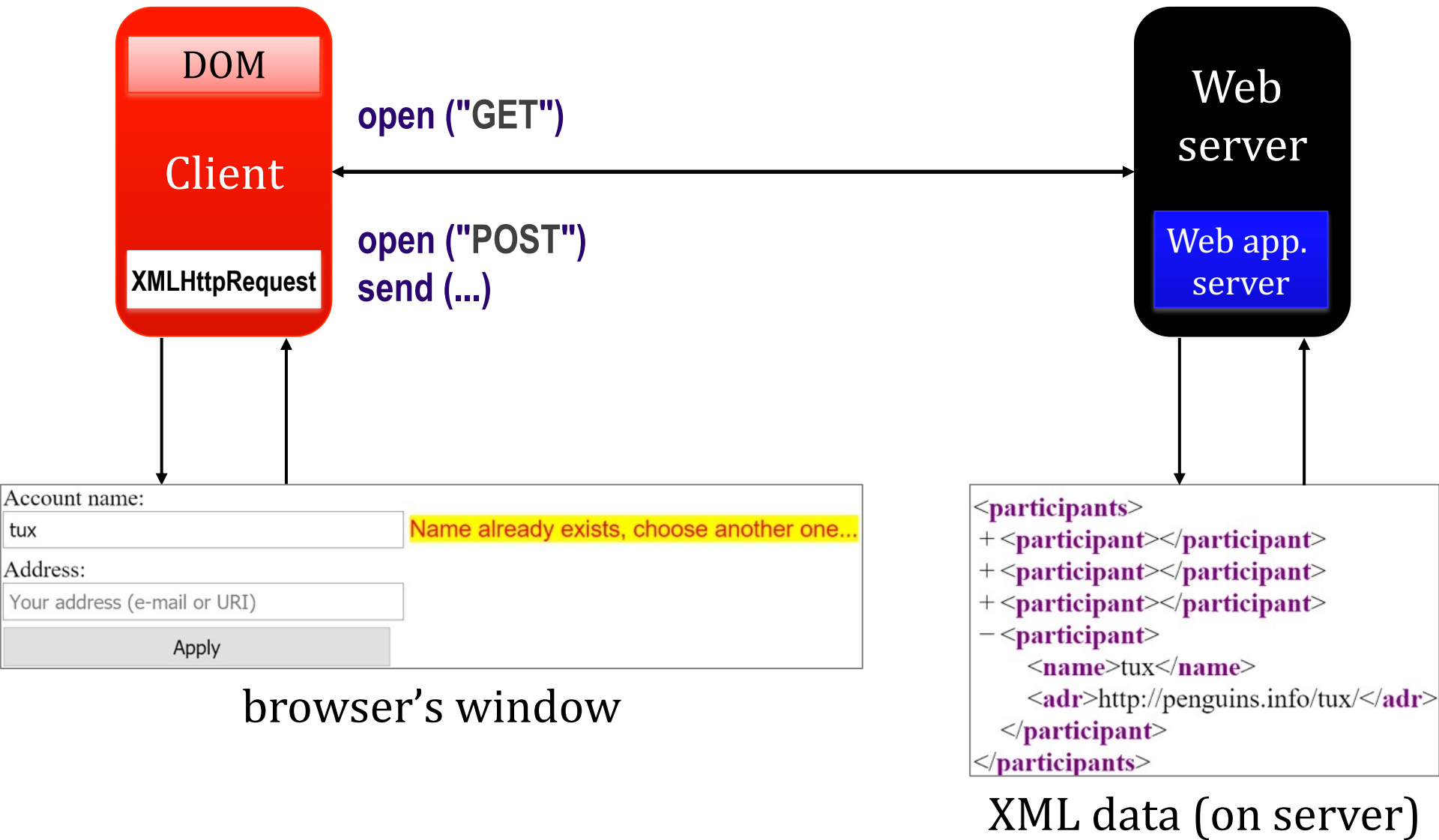
the process of transmitting data to the server (upload)
can have a handler associated via the **upload** property

the upload progress can be tracked based on the features
specified by the **ProgressEvent** interface

setting a **timeout** for making a request
(in milliseconds)

web interaction: ajax – case study

Verifying the existence of a username
in order to create an account
that provides access to a Web application



checking asynchronously if an account exists on server

web interaction: ajax – case study

Checking the existence of a username
in order to create an account
that provides access to a Web application

by handling with DOM the **blur** event, we can detect
– by asynchronously querying the backend – if
the account name entered by the user
in a Web form already exists

web interaction: ajax – case study

Checking the existence of a username
in order to create an account
that provides access to a Web application

the Web application on the server will provide an XML
document modeling the response
to the question: “Does a user exist having that name?”

```

<?php                                // a PHP program, as a Web service, running on server
define ('DOCXML', './particip.xml'); // location of the XML document
// sending content type; here, XML
header ('Content-type: text/xml');

// function that verifies if a participant name already exists
// returns 1 if name exists, 0 otherwise
function checkIfNameExists ($aName) {
    // load data about participants by using SimpleXML
    if (!($xml = simplexml_load_file (DOCXML))) { return 0; }
    // scanning all participants' names found with XPath...
    foreach ($xml->xpath('/participants/participant/name') as $name) {
        // compare names; case insensitive
        if (!strcasecmp($aName, $name)) { return 1; }
    }
    return 0;
}
?>

<response>
    <result><?php echo checkIfNameExists ($_REQUEST['name']); ?></result>
</response>

```



```
<!-- a Web form getting data from user -->
```

```
<form action="add.php" method="get">
```

```
<div>
```

```
<label for="name">Account name:</label>
```

```
<input type="text" name="name" id="name"
```

```
  placeholder="Enter the desired account name"
```

```
  onblur="javascript:signalNameExists (this.value, ")" />
```

```
<!-- a message, initially hidden -->
```

```
<span class="hidden" id="errName">
```

```
  Name already exists, choose another one...
```

```
</span>
```

```
</div>
```

```
<div>
```

```
<label for="adr">Address:</label>
```

```
<input type="text" name="adr" id="adr" />
```

```
</div>
```

```
<input type="submit" value="Apply" />
```

```
</form>
```

```
/* CSS styles */
```

```
.err { /* error message */
```

```
  display: inline;
```

```
  color: red;
```

```
  background-color: yellow;
```

```
  font-family: sans-serif;
```

```
}
```

```
.hidden { /* hidden message */
```

```
  display: none;
```

```
}
```

// a JS program – the classic version – executed by the browser
var request; // encapsulates the HTTP request to the Web server

```
function loadXML (url) { // loads an XML document denoted by 'url'  
    // checking the availability of XMLHttpRequest object  
    if (window.XMLHttpRequest) {  
        request = new XMLHttpRequest (); // there is native support  
    } else  
        if (window.ActiveXObject) { // we can use ActiveX object from MSIE  
            request = new ActiveXObject ("Microsoft.XMLHTTP");  
        }  
    if (request) { // Ajax is supported  
        // set the function that handles data transfer state  
        request.onreadystatechange = handleResponse;  
        // retrieving the document by using GET method  
        request.open ("GET", url, true);  
        request.send (null); // not sending anything to the Web service  
    }  
}
```

```
// handles the change of transfer state
function handleResponse () {
    // verify if the transfer is successful
    if (request.readyState == 4) {
        if (request.status == 200) { // did we get '200 Ok' status code?
            // process received data with DOM
            // (get the root element of the XML document)
            var response = request.responseXML.documentElement;
            var res = response.getElementsByTagName('result')[0].firstChild.data;
            // calling a function that will modify the DOM tree of the Web page
            // according to the response transmitted by the invoked service
            // to signal the (in)existence of the username
            signalNameExists ("", res);
        }
        // eventually, other HTTP status codes (404, 500, etc.) could be handled
    } else {
        alert ("A problem occurred (XML data transfer):\n" +
            request.statusText);
    }
}
```

```
// function that checks the existence of a username
function signalNameExists (name, exist) {
  // do we have an answer?
  if (exist != "") {
    // get via DOM methods the element having the id='errName'
    // to display the error message
    var msg = document.getElementById ('errName');
    // change the style (in case of error, the CSS properties
    // in the class 'eroare' will be applied, otherwise the text will be hidden)
    msg.className = exist == 1 ? 'err' : 'hidden';
  } else {
    // there is no answer set, will check asynchronously
    // by sending a request to the server
    loadXML ('verify.php?name=' + name);
  }
}
```

study the example in
the archive

A new participant

Account name:

Name already exists, choose another one...

Address:

Apply

Status	Method	File	Domain	Cause	Headers	Cookies	Params	Response
200	GET	verify.php?name=tux	localhost	JS xhr	Response payload			
200	GET	verify.php?name=busaco	localhost	JS xhr	1	<response>		
200	GET	verify.php?name=tux	localhost	JS xhr	2	<result>1</result>		
					3	</response>		

the user provides an account name; via Ajax, (s)he will be notified if this name already exists, according to the XML response sent by the Web service

HTTP request using the URL: <http://web.info/verify.php?name=tux>

XML response: `<response><result>1</result></response>`

0 = does not exist

web interaction: **ajax**

Complementary, **Fetch** can be used
(HTML Living Standard, 1 April 2026)

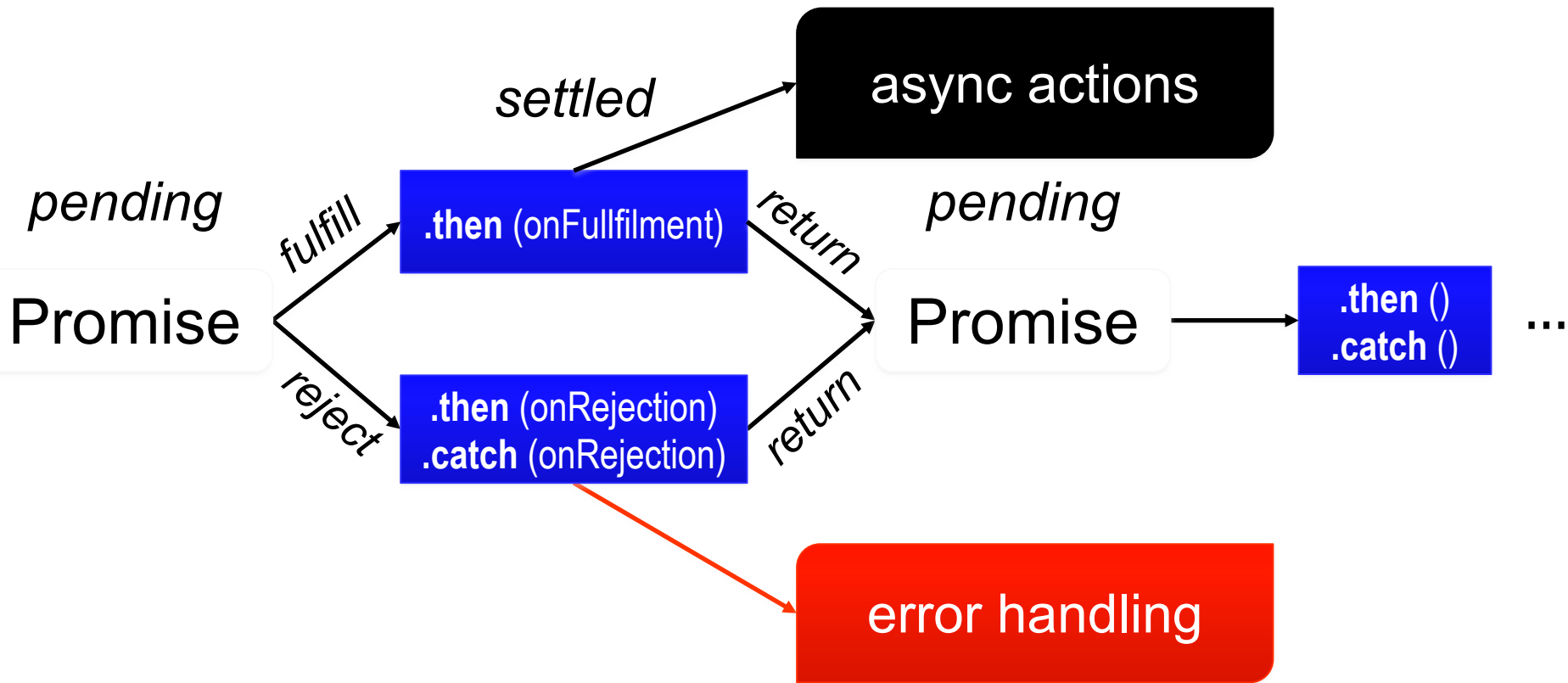
API provided by any modern web browser for
specifying access to data (resource representations)
after initiating an HTTP request and
getting a response from the server

fetch.spec.whatwg.org

web interaction: ajax

The new **Fetch API** implemented by the browser is based on the concept of *promise* $\equiv$ a result that could be offered after executing an asynchronous operation

represents an operation that has not completed yet, but is expected in the future



developer.mozilla.org/Web/JavaScript/Guide/Using_promises
exploringjs.com/es6/ch_promises.html
web.dev/promises/

case study: RandomAjax (Fetch)

```
<div id="numbers">[Wait, please...]</div>
```

```
// A JS program that asynchronously get (by using the Fetch API) a sequence of numbers  
by random.org. See also "Introduction to fetch()": https://developers.google.com/web/updates  
/2015/03/introduction-to-fetch
```

```
const URL = 'https://www.random.org/sequences/?min=1&max=33&col=1&format=plain&rnd=new';
```

```
function status(response) { // using promises to handle the returned HTTP status code  
  if (response.status >= 200 && response.status < 300) {  
    return Promise.resolve(response)  
  } else {  
    return Promise.reject(new Error(response.statusText))  
  }  
}
```

```
fetch(URL)  
  .then(status) // checking if data was successfully received  
  .then((response) => response.text()) // transforming received data into a string  
  .then(function(response) { // processing the number sequence  
    // substituting white spaces with comma and  
    // putting the content into the HTML element identified by 'numbers'  
    var numbers = document.getElementById('numbers');  
    numbers.textContent = response.trim().replace(/\W+/g, ', ');  
  })  
  .catch(function(error) { // an error occurred :(  
    numbers.textContent = 'An error occurred: ' + error;  
  });
```

HTML

```
1 #numbers {  
2   font-family: monospace;  
3   font-size: 1em;  
4   width: 12em;  
5 }  
6
```

```
28, 25, 7, 33, 23,  
22, 14, 21, 5, 1,  
18, 2, 4, 13, 12, 6,  
20, 17, 19, 24, 16,  
31, 11, 29, 10, 3,  
8, 26, 30, 27, 15,  
32, 9
```

gets asynchronously
using **Fetch API**
a sequence of random
numbers generated by
random.org – sent as
plain text

jsfiddle.net/busaco/a2q9regd/

We use promises to perform processings depending on the received HTTP status code

```
function status(response) {  
  if (response.status >= 200 && response.status < 300) {  
    // the request can be fulfilled  
    return Promise.resolve(response)  
  } else {  
    // the request was rejected  
    return Promise.reject(new Error(response.statusText))  
  }  
}
```

```
let numbers = document.getElementById ('numbers');

fetch (URL)
  .then (status) // check if data was successfully received
  // transform the object of the answer into a string
  .then (response => response.text ())
  // process the sequence of numbers
  .then (response => {
    // replace whitespaces with comma and put the content
    // into the HTML element identified by 'numbers'
    numbers.textContent = response.trim().replace (/W+/g, ', ');
  })
  .catch (error => { // an error occurred :(
    numbers.textContent = 'An error occurred: ' + error;
  });
```

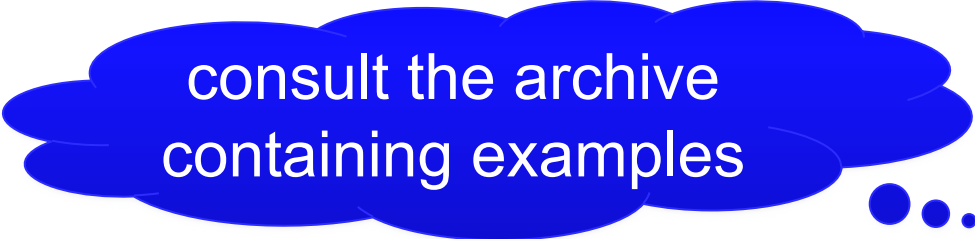
case study: PostJSON

Scenario – modern JS implementation (ES6):
get characters from the client – inserted in <textarea> –
and send them asynchronously via POST to a Web
application that runs on a server which sends them back

data transferred in JSON format
{ “**key**”: “*character*”, “**date**”: “*seconds*” }

handle the **keypress** event to capture the keys
pressed by the user

```
// ES6 code interpreted by the Web browser
// handle the key press event
const trateazaEveniment = ev => {
  // the message sent to the server via POST
  // when the event occurs
  let msg = `{ "tasta": "${String.fromCharCode (ev.charCode)}",
               "data": "${Date.now()}" }`;
  // create a POST request, encapsulated in a Request object
  let request = new Request ('/ajax/post.php', {
    method: 'POST',
    // convert JSON data into string
    body: JSON.stringify (msg),
    // no header fields
    headers: {}
  });
```



consult the archive
containing examples

...

```
fetch (request) // we promise to execute the code, by sending the request...
.then (response => { // check if we received JSON data from the server
    let contentType = response.headers.get ('Content-Type');
    if (contentType && contentType.includes('application/json')) {
        return response.json ();
    };
    throw new TypeError ("Datele primite nu-s JSON");
})
.then (json => { // process date
    // create via DOM a text node which indicates the key that was pressed
    let elem = document.createTextNode (json.tasta);
    document.getElementById ('tasteApasate').appendChild (elem);
    // log received data into browser's console
    console.log
        (`Date JSON primite: tasta=${json.tasta}, data=${Date(json.data)}`);
})
};
// via DOM, handle keypress event
document.addEventListener ('keypress', trateazaEveniment);
```

// post.php – PHP program which receives JSON data
// sent via POST by a client and sends them back (echo)

```
function eJSONValid ($sir) { // check if JSON data is correct
    json_decode ($sir);
    return json_last_error () == JSON_ERROR_NONE;
}
```

```
// get from standard input the data sent by the client (raw data)
// (here, the one in a POST request)
$date = trim (file_get_contents ("php://input"));
```

```
if (eJSONValid ($date)) { // if ok, send the data back
    header ("Content-type: application/json");
    echo json_decode ($date);
} else
    die ("Date incorecte");
```

the Web application (service) invoked on the server

Redare taste apasate (trimise si unei aplicatii pe server)

Web

Taste: Web

 Inspector

 Console

 Debugger

 Performance

 Network





 Filter URLs







Disable Cache

No Throttling



All

HTML

CSS

JS

XHR

Fonts

Images

Media

WS

Other

St...	Met...	File	Initiator	Transferred	Size	Headers	Cookies	Request	Response
200	POST	post.pl	post-chars.html:26 (fetch)	228 B	41 B	Filter properties			
200	POST	post.pl	post-chars.html:26 (fetch)	228 B	41 B	JSON		Text	Raw
200	POST	post.pl	post-chars.html:26 (fetch)	228 B	41 B	1	{ "tasta": "W", "data": "1620920514168" }		

 Inspector

 Console

 Debugger

 Performance

 Network





 Filter Output

Errors

Warnings

Logs

Info

Debug

CSS

XHR

Requests



XHR

POST

http://localhost:8080/post.php

[HTTP/1.1 200 OK 0ms]

Date JSON primite: tasta=W, data=Thu May 13 2021 18:41:54 GMT+0300 (Eastern European Summer Time)

post-chars.html:41:14

XHR

POST

http://localhost:8080/post.php

[HTTP/1.1 200 OK 0ms]

Date JSON primite: tasta=e, data=Thu May 13 2021 18:41:54 GMT+0300 (Eastern European Summer Time)

post-chars.html:41:14

XHR

POST

http://localhost:8080/post.php

[HTTP/1.1 200 OK 0ms]

Date JSON primite: tasta=b, data=Thu May 13 2021 18:41:56 GMT+0300 (Eastern European Summer Time)

post-chars.html:41:14

The screenshot displays the Chrome DevTools Network tab. The top toolbar includes icons for Inspector, Console, Debugger, Network, and DOM. The Network tab is active, showing a list of requests. The 'XHR' filter is selected. The list of requests includes various methods (OPTIONS, POST, GET) to different domains (peoplestackwebexpe..., peoplestack-pa.client..., contacts.google.com, play.google.com, mail.google.com, people-pa.clients6.g...). The selected request is a GET request to 'people-pa.clients6.g...' with a status of 200. The response is a JSON object. The right pane shows the 'Response' tab with the JSON data. A text overlay is present over the lower right portion of the screenshot.

Stat...	Method	Domain	File	Initiator	Type	Transferred	Size
200	OPTIONS	peoplestackwebexpe...	GetExperimentFlags	xhr	html	707 B	0 B
200	OPTIONS	peoplestack-pa.client...	GetAssistiveFeatures	xhr	html	707 B	0 B
200	POST	peoplestackwebexpe...	GetExperimentFlags	m=_b,_tp:601 (xhr)	json	1.24 kB	594 B
200	POST	peoplestack-pa.client...	GetAssistiveFeatures	m=_b,_tp:601 (xhr)	json	1.23 kB	83 B
200	POST	contacts.google.com	batchexecute?rpcids=wiD	m=_b,_tp:601 (xhr)	json	1.95 kB	802 B
200	POST	contacts.google.com	batchexecute?rpcids=wiD	m=_b,_tp:601 (xhr)	json	2.07 kB	1.08 kB
200	POST	play.google.com	log?format=json&hasfast	rs=AA2YrTue4Gb...	NS_ERROR_...		
200	GET	mail.google.com	/mail/u/0/?ui=2&ik=	m=b:1138 (xhr)	html	4.42 kB	43 B
200	POST	mail.google.com	/mail/u/0/?ui=2&ik=16e4	m=b:1138 (xhr)	js	983 B	224 B
200	OPTIONS	people-pa.clients6.g...	lookup?context.clientVers	xhr	html	764 B	0 B
200	GET	people-pa.clients6.g...	lookup?context.clientVers	cb=gapi.loaded_...	json	2.56 kB	7.24 kB
200	POST	mail.google.com	s?hl=en&c=17	m=rn4kU,yWJZb...	json	12.48 kB	40.40 kB
200	OPTIONS	clients6.google.com	files?q=gmail.messageld=	xhr			
200	GET	clients6.google.com	files?q=gmail.messag	cb=gapi.loaded			
200	POST	mail.google.com	s?hl=en&c=18	m=rn4kU,yWJZb...			
200	POST	play.google.com	log?format=json&hasfast	m=_b,_tp:601			
200	POST	play.google.com	log?format=json&hasfast	m=_b,_tp:601			
200	POST	mail.google.com	el2?hl=en&c=19	m=rn4kU,yWJZb...	json	918 B	2 B

inspecting transferred data using the Web developer tools provided by the browser

```
Object {  
  personId: "c2926004446151057113",  
  fingerprint: "%EgYBAGMJCygaBQECBwgLlgwwK1k5L2RYR0ZOdz0=",  
  metadata: {  
    lastUpdateTimeMicros: "1647583043926",  
    model: "CONTACT_CENTRIC",  
    contactId: [...],  
  },  
  contactId: ["2926004446151057113"]  
}
```

⚠ data transfer can be monitored (+intercepted) via dedicated tools

web interaction: ajax

Ajax offers the premises for asynchronous invocation of REST Web (micro-)services

web interaction: ajax

Ajax offers the premises for asynchronous invocation of REST Web (micro-)services

using various representations of transferred data:

POX (Plain Old XML)

JSON (JavaScript Object Notation)

AHAH (Asynchronous HTML and HTTP)

plain text

web interaction: ajax



Alternative solutions:
adopting various HTML5 technologies

server-sent events

developer.mozilla.org/docs/Web/API/Server-sent_events

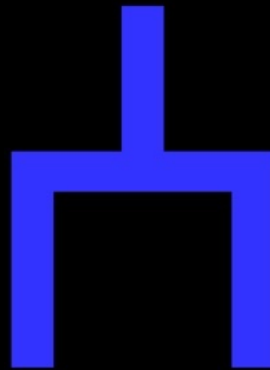
WebSockets (living standard – 15 March 2026)

websockets.spec.whatwg.org

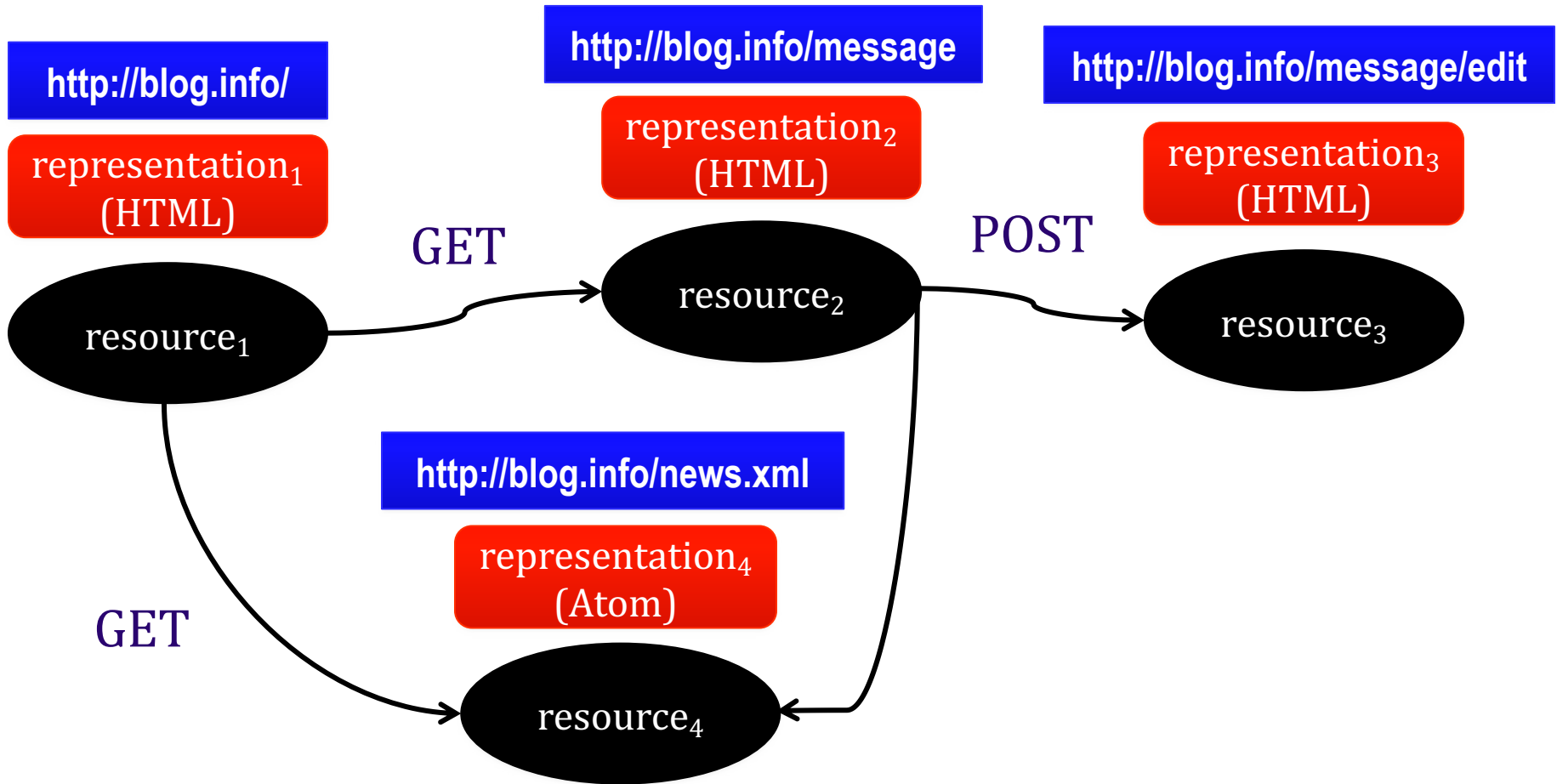
developer.mozilla.org/docs/Web/API/WebSockets_API

summary

XML/HTML access via **DOM**



characterization, specification, interfaces, XPath,
events, data- attributes, Ajax, examples



next episode: **Web services**